
전자항법과 GPS

13주차

공간정보 프로그래밍 실습

임 호 용

imyong7@hanmail.net



전자항법과 GPS

1. 공간정보 프로그래밍

1-1. Node.js & VSCode

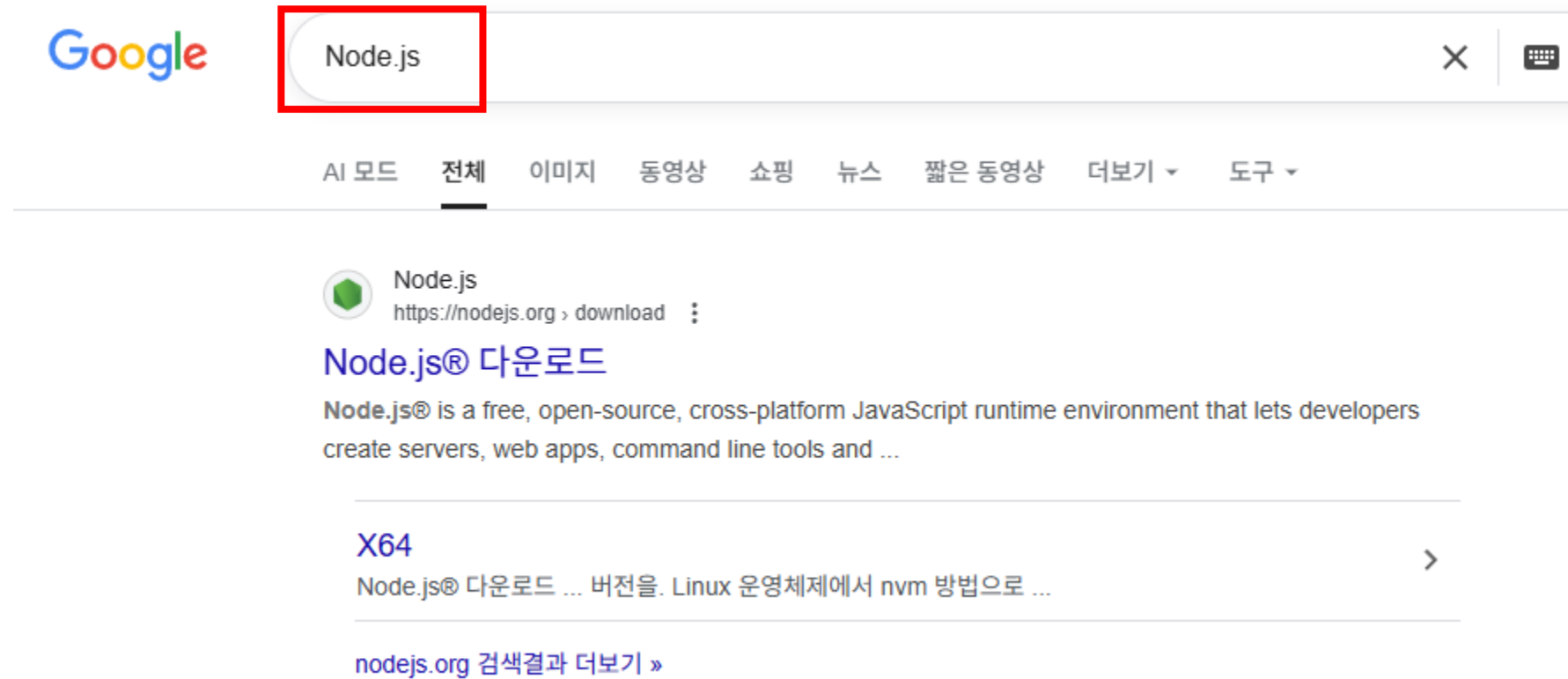
1-2. Node.js & Express & PostgreSQL

1-3. Etc. Tools

전자항법과 GPS

1. 공간정보 프로그래밍

1-1. *Node.js & VSCode*







Node.js® 다운로드

Node.js® v24.11.1 (LTS) 를 Windows 환경에서 Docker 방식으로 npm 를(을) 사용해
설치하세요.

정보 Want new features sooner? Get the [latest Node.js version](#) instead and try the latest improvements!

```
1 # Docker는 각 운영 체제별로 설치 지침을 제공합니다.
2 # 공식 문서는 https://docker.com/get-started/에서 확인하세요.
3
4 # Node.js Docker 이미지를 풀(Pull)하세요:
5 docker pull node:24-alpine
6
7 # Node.js 컨테이너를 생성하고 셸 세션을 시작하세요:
8 docker run -it --rm --entrypoint sh node:24-alpine
9
10 # Verify the Node.js version:
11 node -v # Should print "v24.11.1".
12
13 npm 버전 확인:
14 npm -v # 11.6.2가 출력되어야 합니다.
```

PowerShell  클립보드에 복사

Docker는 컨테이너화 플랫폼입니다. 문제가 발생하면 [Docker's 웹사이트](#) >를 방문하세요.

또는 x64 아키텍처가 실행 중인 Windows 환경에서 미리 빌드된 Node.js®를 다운로드하세요.

 Windows 설치 프로그램 (.msi)

 Standalone Binary (.zip)

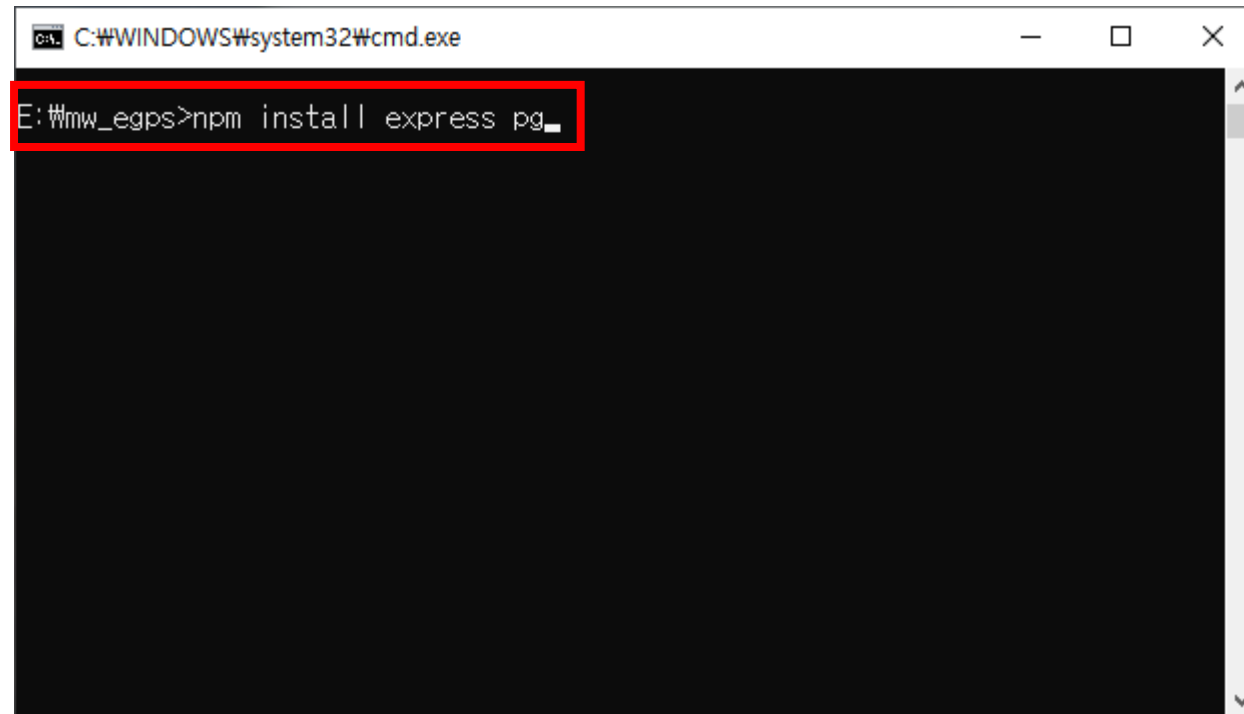
```
C:\WINDOWS\system32\cmd.exe
E:\mw_egps>dir/w
E 드라이브의 볼륨: FireCuda Gaming Dock
볼륨 일련 번호: B030-05F3

E:\mw_egps 디렉터리
[.]  [...]
                0개 파일                0 바이트
                2개 디렉터리  1,579,773,108,224 바이트 남음
E:\mw_egps>npm init -y
```

```
C:\WINDOWS\system32\cmd.exe
E:\mw_egps>npm init -y
Wrote to E:\mw_egps\package.json:

{
  "name": "mw_egps",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo W\"Error: no test specifiedW\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}

E:\mw_egps>
```



```
C:\WINDOWS\system32\cmd.exe
E:\mw_egps>npm install express pg
```



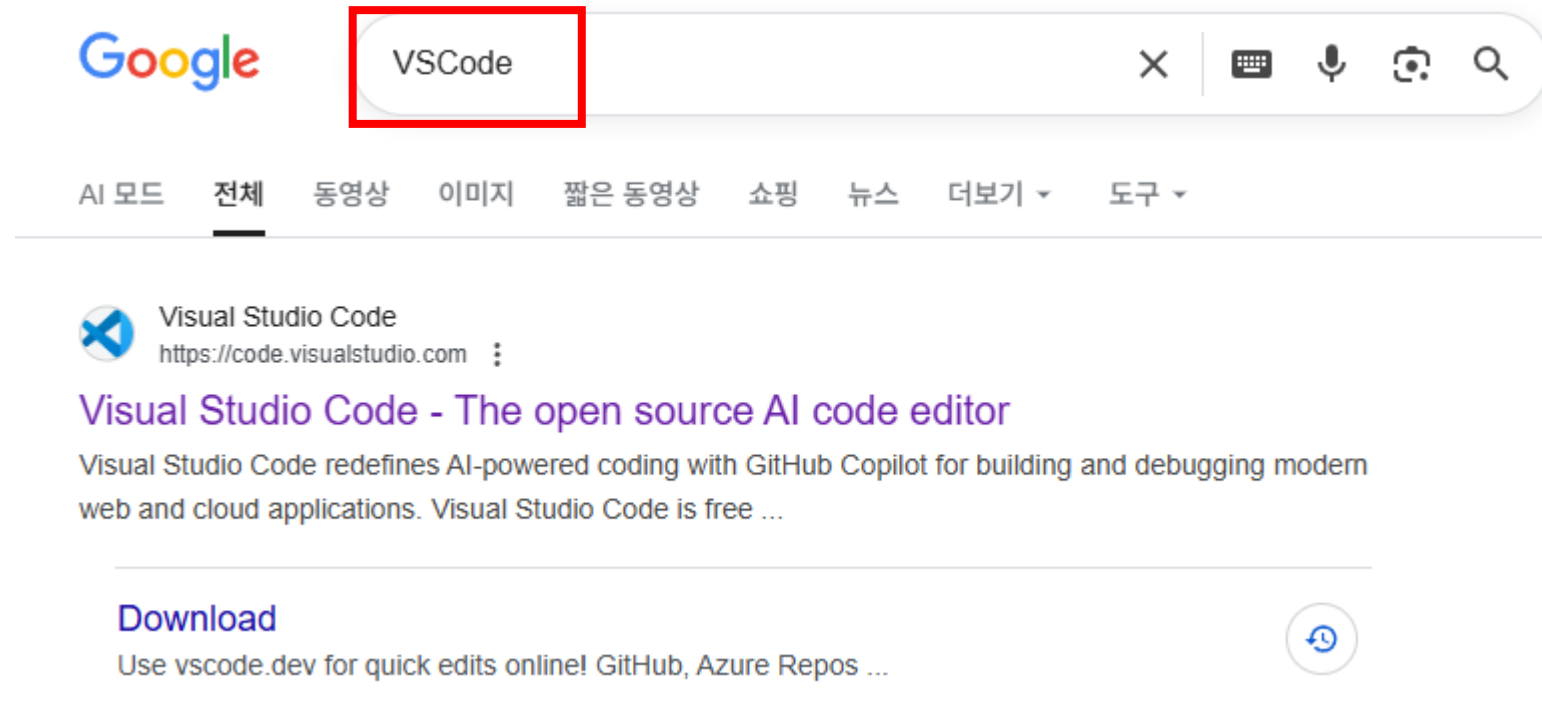
```
C:\WINDOWS\system32\cmd.exe
npm WARN EBADENGINE }
npm WARN EBADENGINE Unsupported engine {
npm WARN EBADENGINE   package: 'send@1.2.0',
npm WARN EBADENGINE   required: { node: '>= 18' },
npm WARN EBADENGINE   current: { node: 'v16.15.0', npm: '8.5.5' }
npm WARN EBADENGINE }
npm WARN EBADENGINE Unsupported engine {
npm WARN EBADENGINE   package: 'serve-static@2.2.0',
npm WARN EBADENGINE   required: { node: '>= 18' },
npm WARN EBADENGINE   current: { node: 'v16.15.0', npm: '8.5.5' }
npm WARN EBADENGINE }

added 79 packages, and audited 80 packages in 18s

19 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

E:\mw_egps>
```





[Version 1.106](#) is now available! Read about the new features and fixes from October.



Download Visual Studio Code

Free and built on open source. Integrated Git, debugging and extensions.



↓ Windows
Windows 10, 11

User Installer [x64](#) [Arm64](#)
System Installer [x64](#) [Arm64](#)
.zip [x64](#) [Arm64](#)
CLI [x64](#) [Arm64](#)



↓ .deb
Debian, Ubuntu

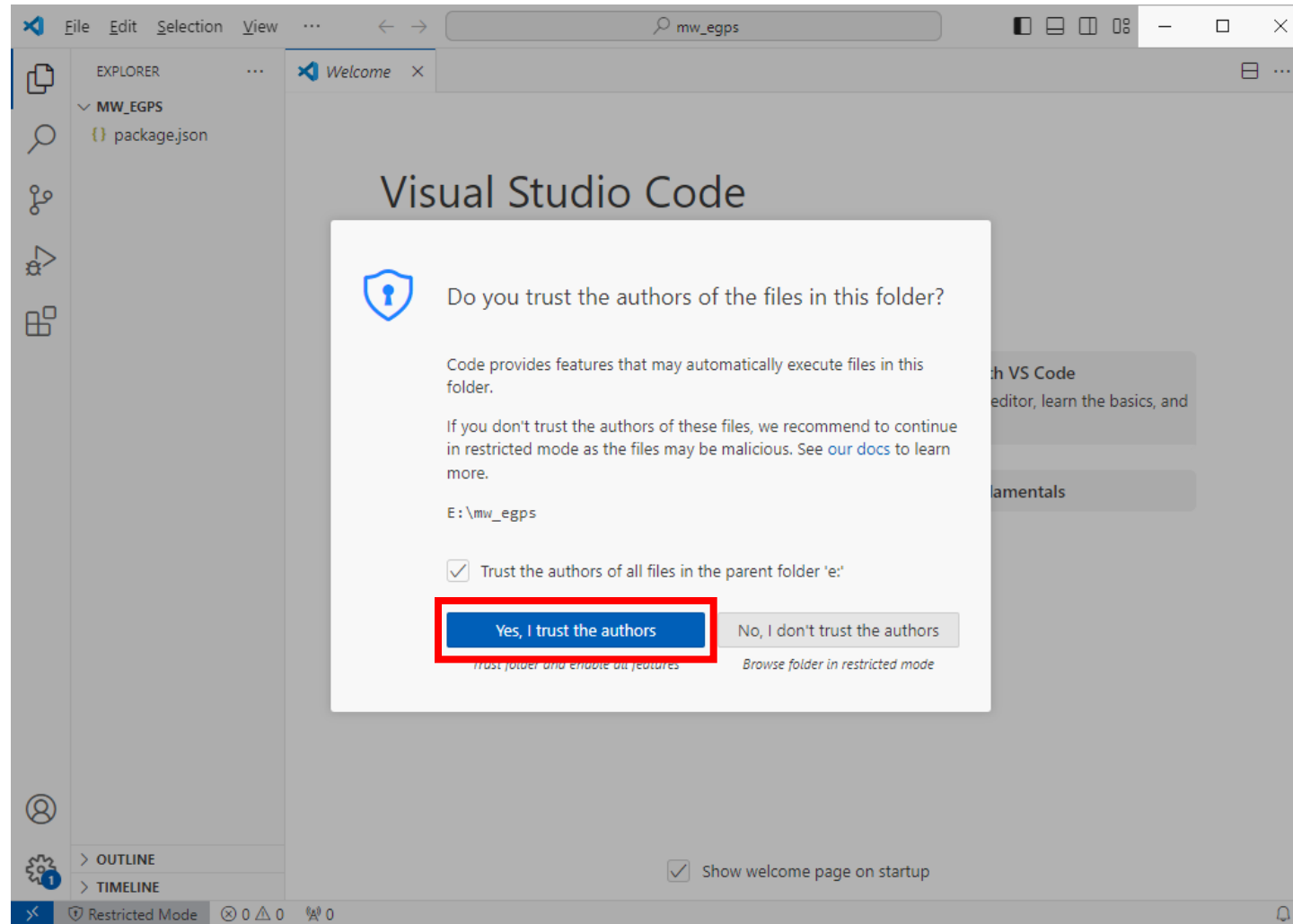
↓ .rpm
Red Hat, Fedora, SUSE

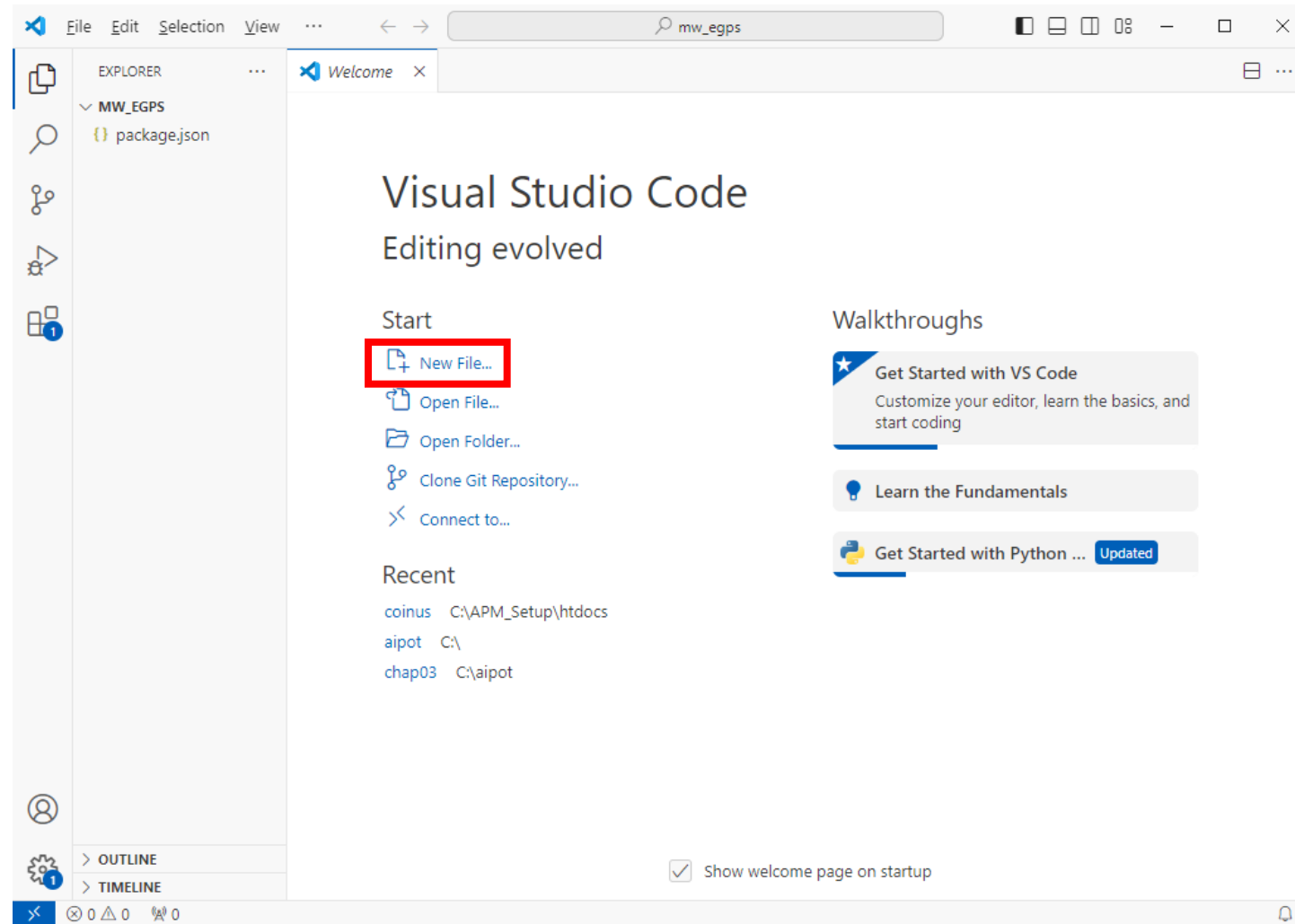
.deb [x64](#) [Arm32](#) [Arm64](#)
.rpm [x64](#) [Arm32](#) [Arm64](#)
.tar.gz [x64](#) [Arm32](#) [Arm64](#)
Snap [Snap Store](#)
CLI [x64](#) [Arm32](#) [Arm64](#)

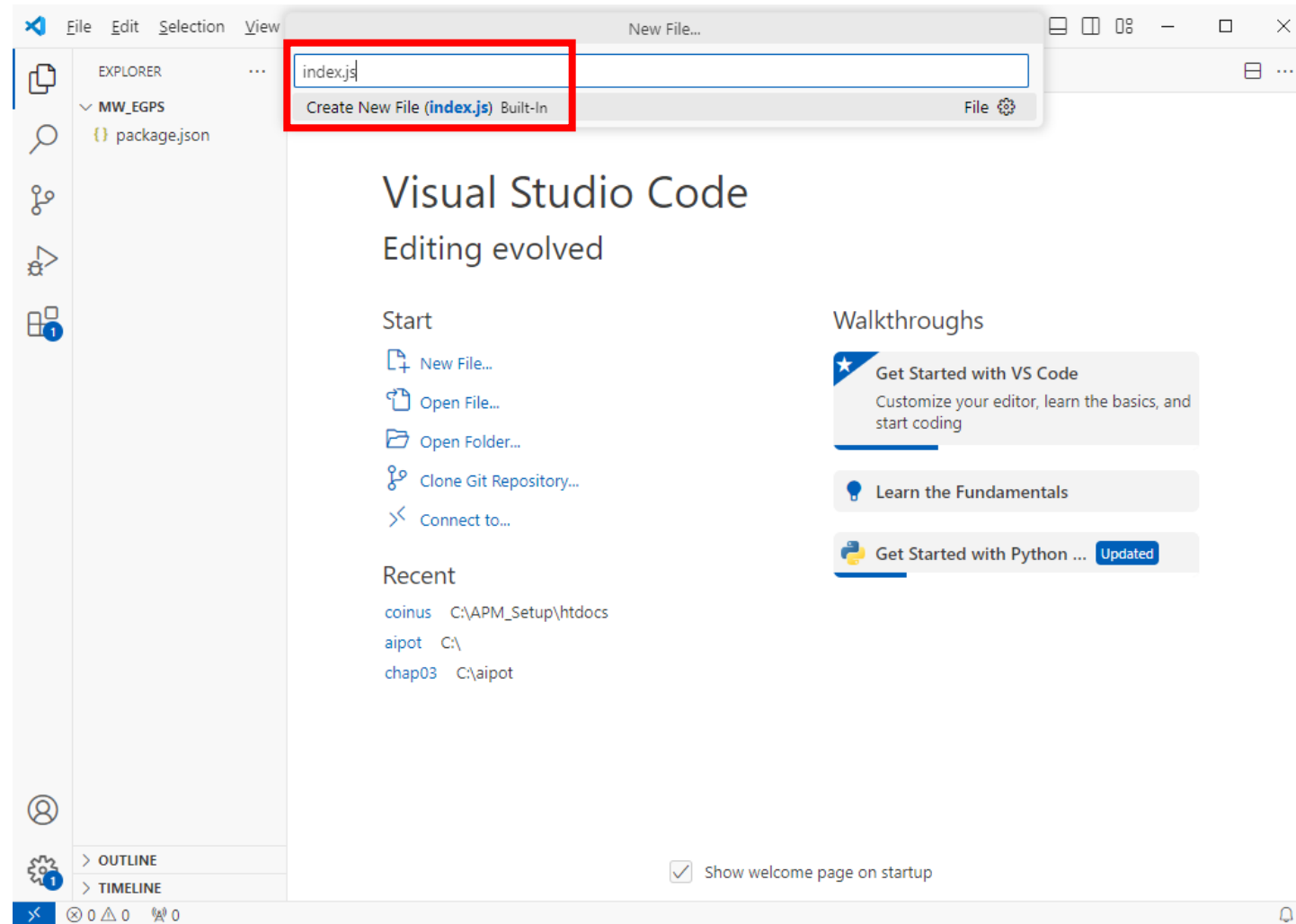


↓ Mac
macOS 11.0+

.zip [Intel chip](#) [Apple silicon](#) [Universal](#)
CLI [Intel chip](#) [Apple silicon](#)

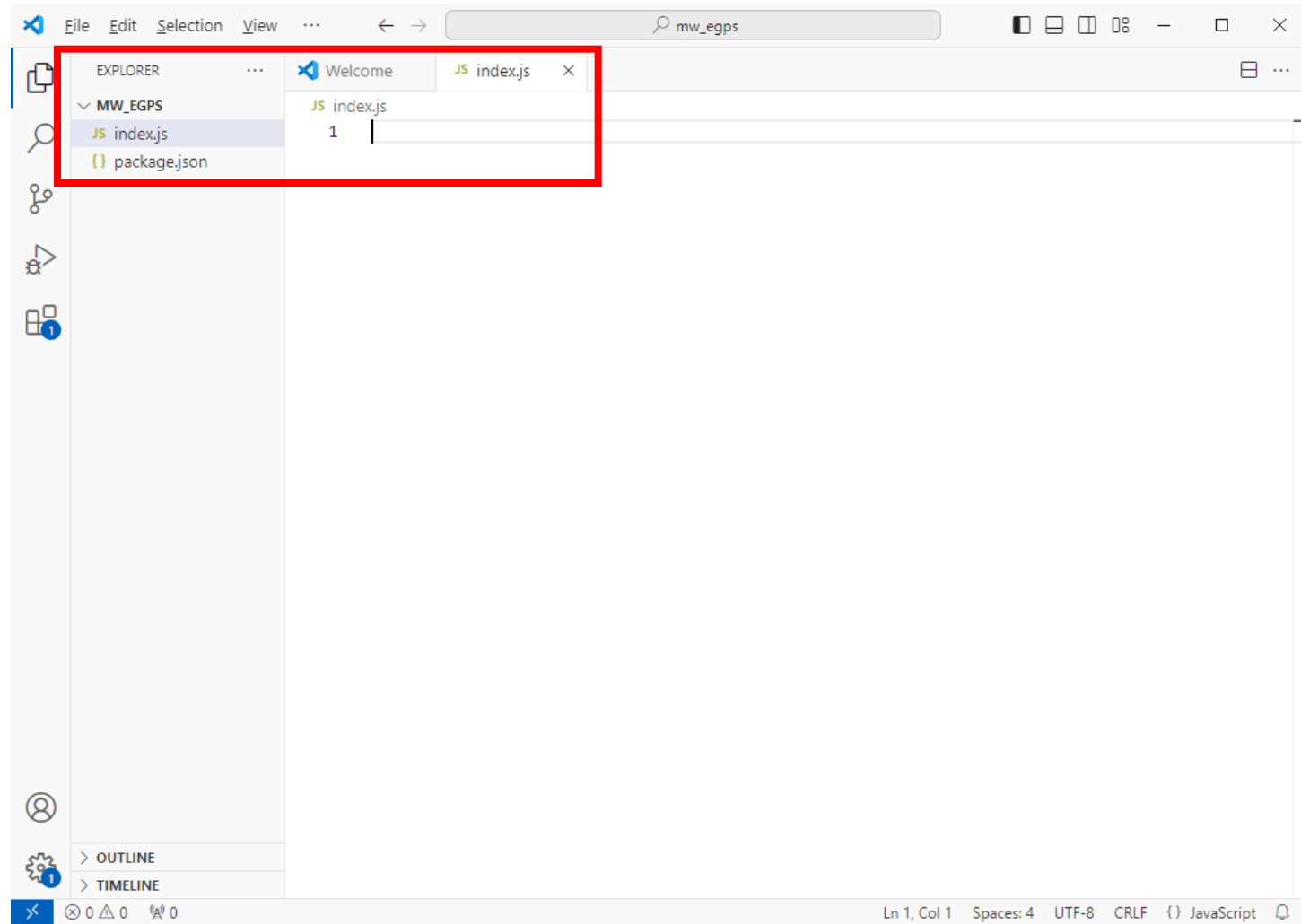






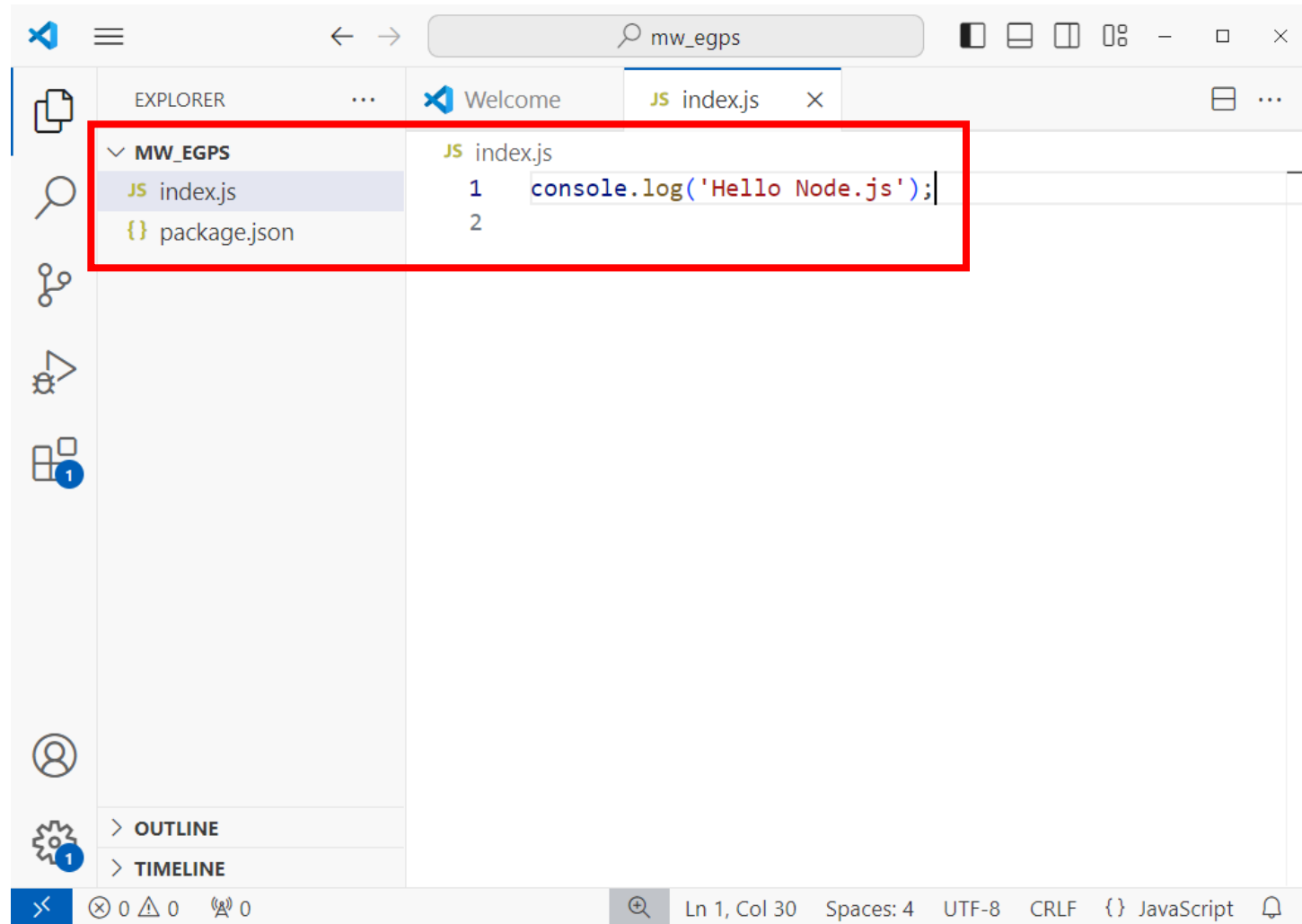
1. 공간정보 프로그래밍

VSCode



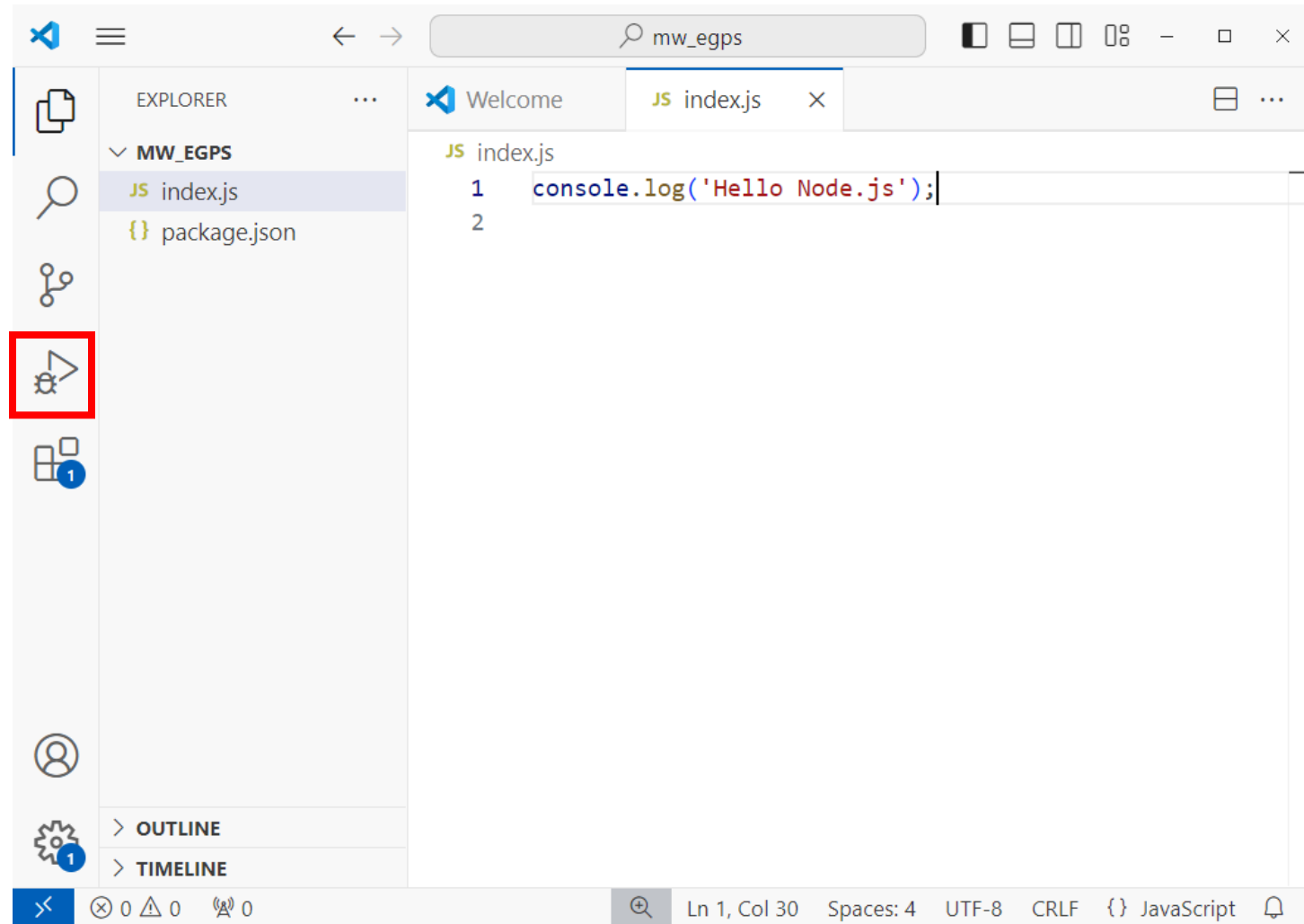
1. 공간정보 프로그래밍

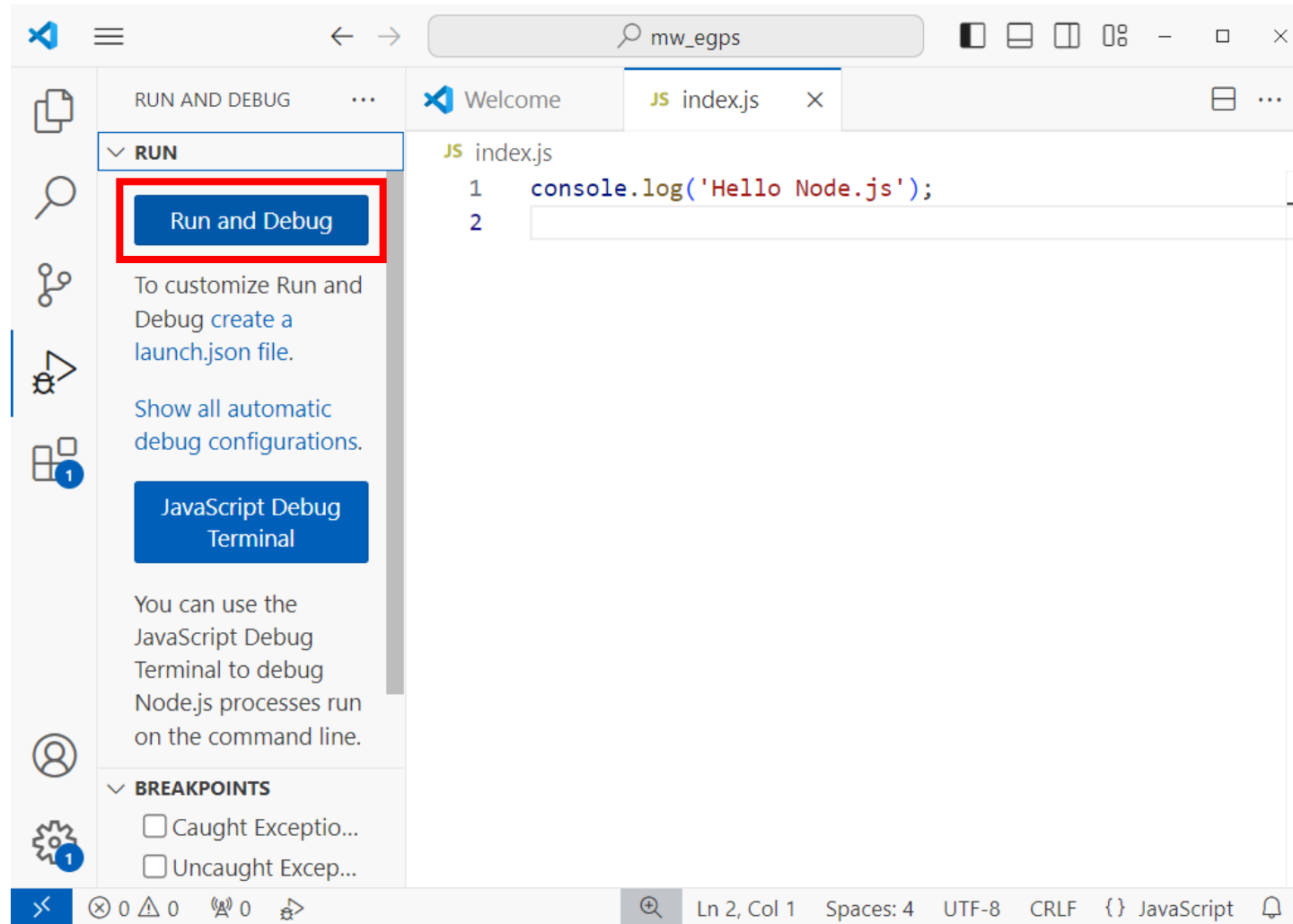
VSCode

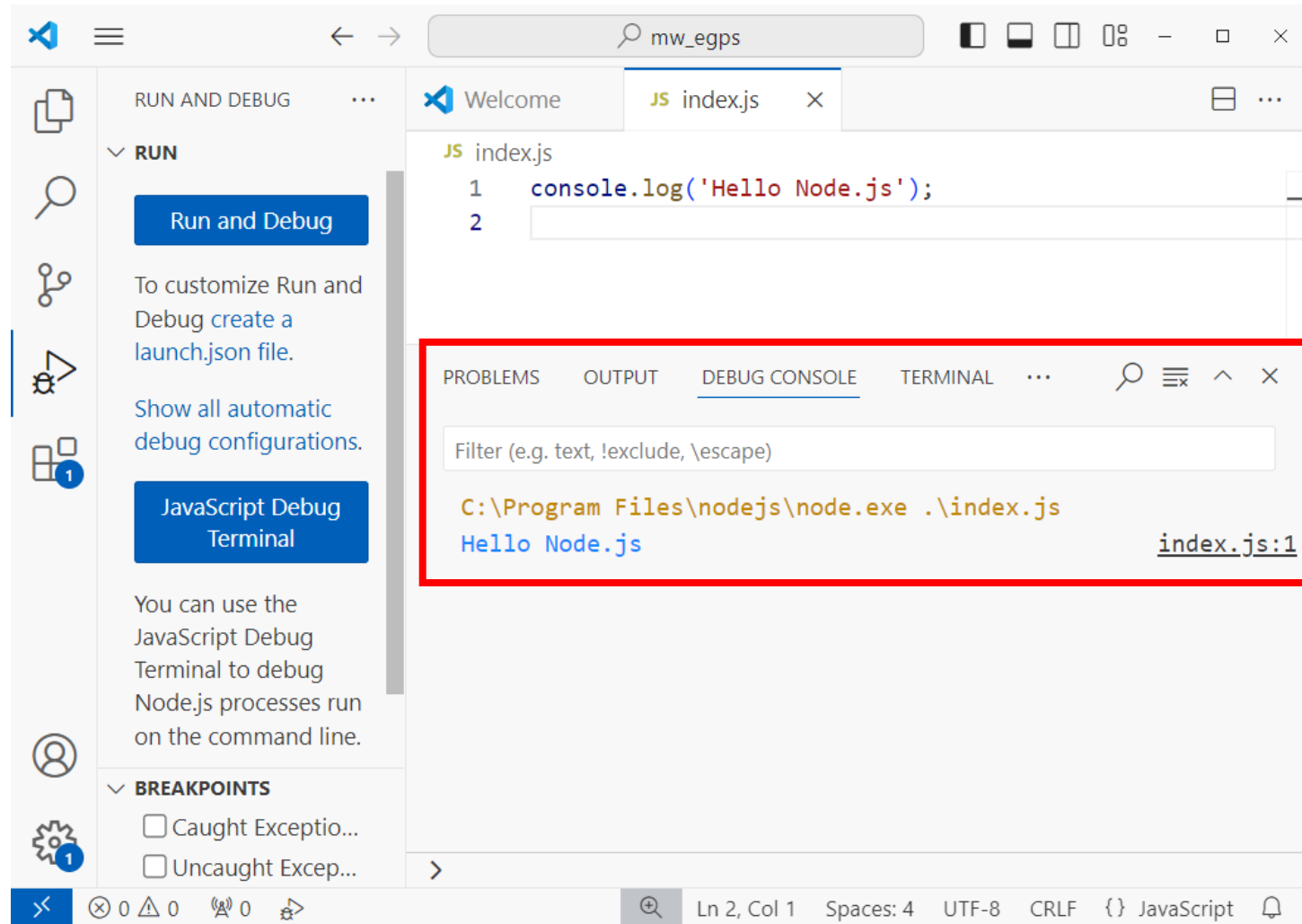


1. 공간정보 프로그래밍

VSCode



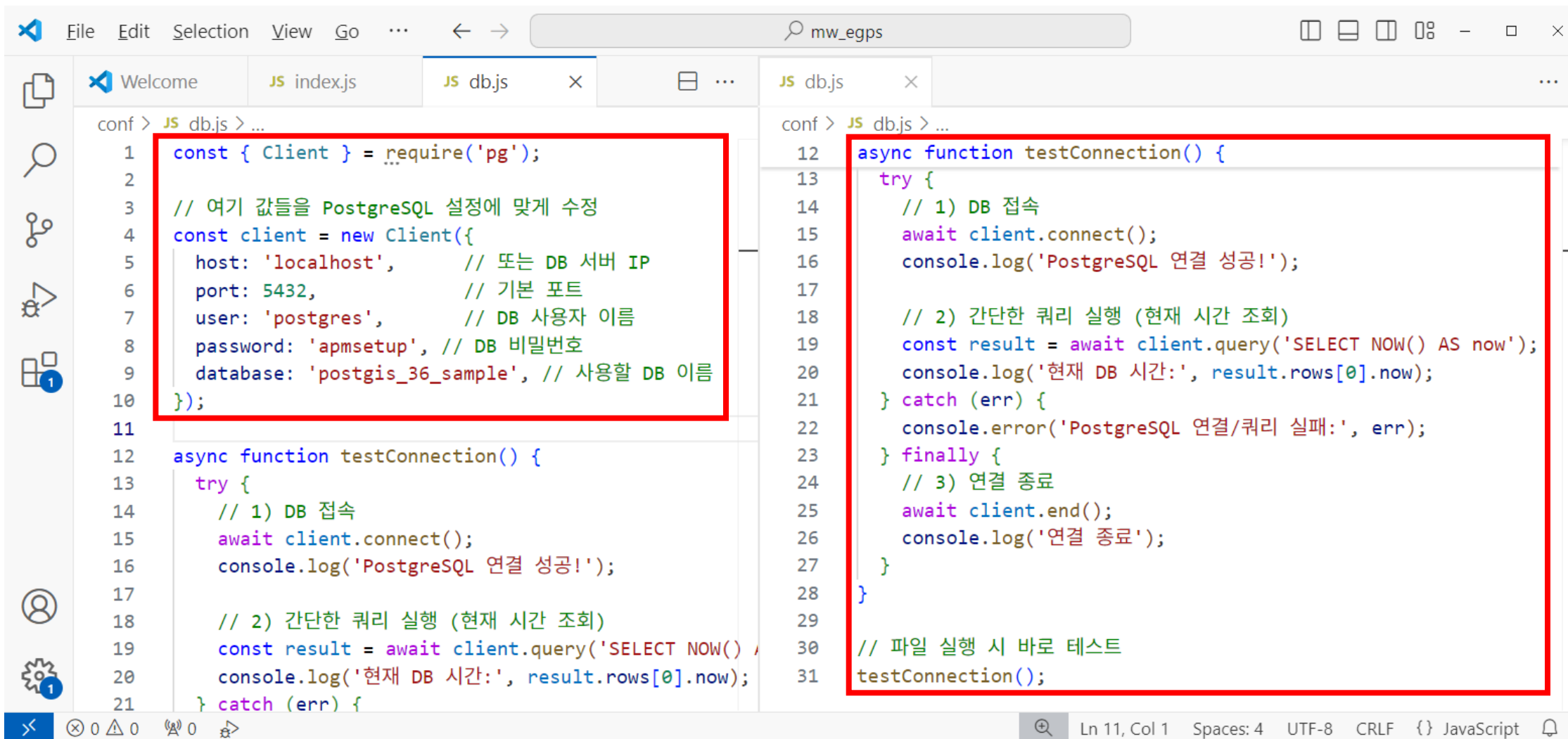




전자항법과 GPS

1. 공간정보 프로그래밍

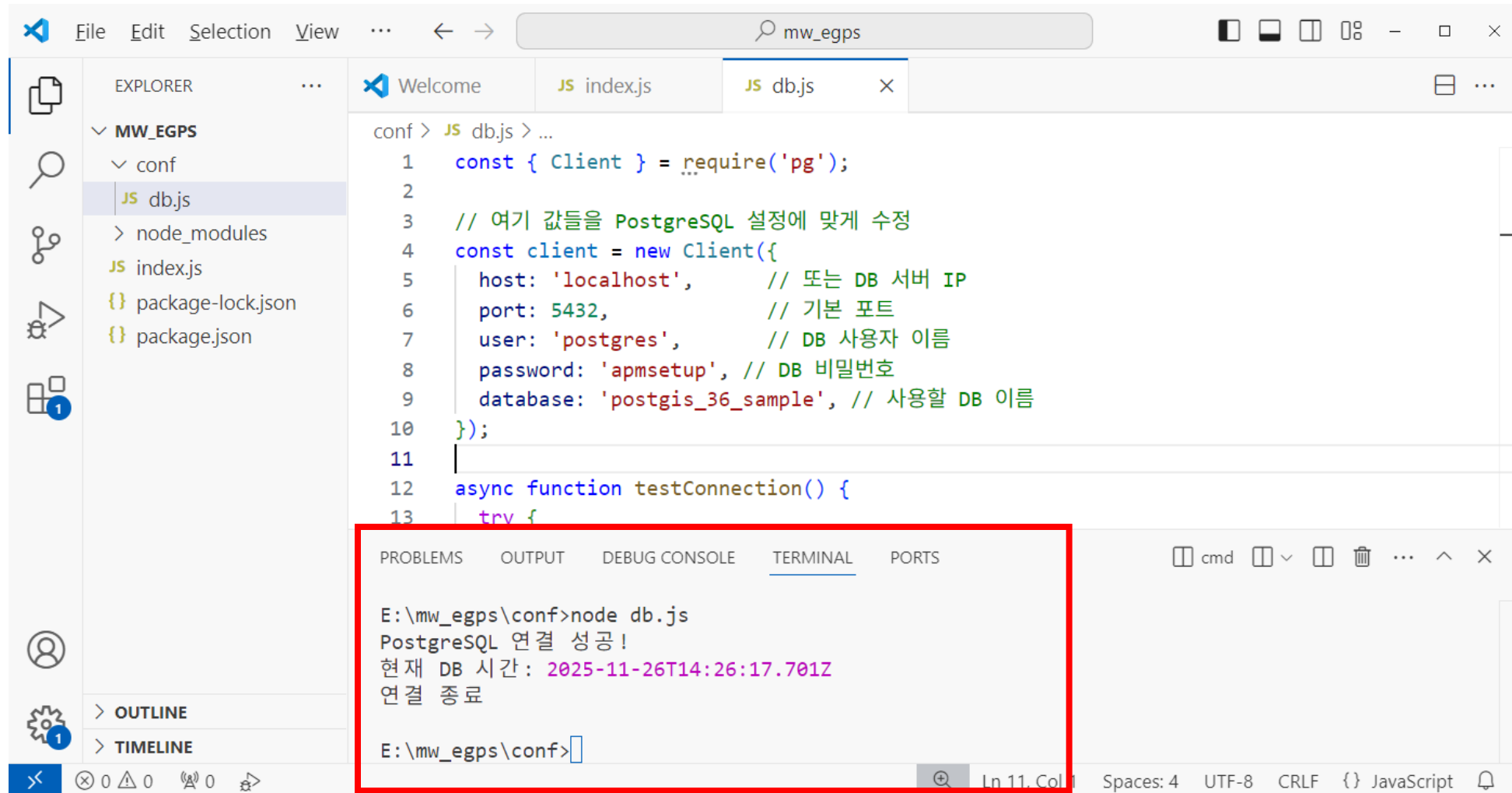
1-2. *Node.js & Express & PostgreSQL*



The image shows a VS Code editor window with two files open: `index.js` and `db.js`. The `db.js` file is the active editor, showing code for connecting to a PostgreSQL database. The code is divided into two sections, each highlighted with a red box.

```
conf > JS db.js > ...
1  const { Client } = require('pg');
2
3  // 여기 값들을 PostgreSQL 설정에 맞게 수정
4  const client = new Client({
5    host: 'localhost',      // 또는 DB 서버 IP
6    port: 5432,             // 기본 포트
7    user: 'postgres',      // DB 사용자 이름
8    password: 'apmsetup',  // DB 비밀번호
9    database: 'postgis_36_sample', // 사용할 DB 이름
10  });
11
12  async function testConnection() {
13    try {
14      // 1) DB 접속
15      await client.connect();
16      console.log('PostgreSQL 연결 성공!');
17
18      // 2) 간단한 쿼리 실행 (현재 시간 조회)
19      const result = await client.query('SELECT NOW() AS now');
20      console.log('현재 DB 시간:', result.rows[0].now);
21    } catch (err) {
22      console.error('PostgreSQL 연결/쿼리 실패:', err);
23    } finally {
24      // 3) 연결 종료
25      await client.end();
26      console.log('연결 종료');
27    }
28  }
29
30  // 파일 실행 시 바로 테스트
31  testConnection();
```

The status bar at the bottom indicates the current position is Line 11, Column 1, with 4 spaces, UTF-8 encoding, CRLF line endings, and JavaScript language mode.



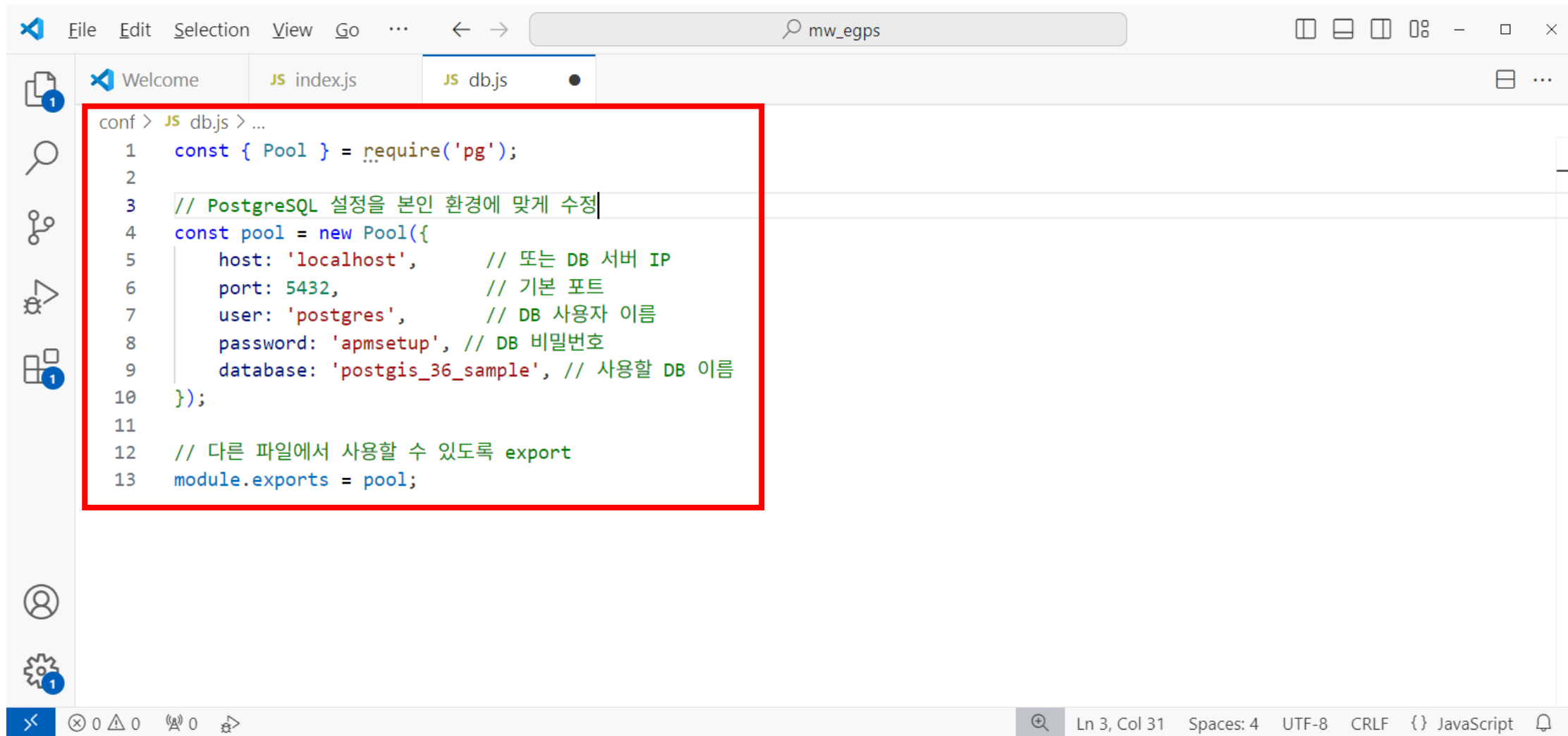
```
conf > JS db.js > ...
1  const { Client } = require('pg');
2
3  // 여기 값들을 PostgreSQL 설정에 맞게 수정
4  const client = new Client({
5    host: 'localhost',      // 또는 DB 서버 IP
6    port: 5432,             // 기본 포트
7    user: 'postgres',      // DB 사용자 이름
8    password: 'apmsetup',  // DB 비밀번호
9    database: 'postgis_36_sample', // 사용할 DB 이름
10  });
11
12  async function testConnection() {
13    try {
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
E:\mw_egps\conf>node db.js
PostgreSQL 연결 성공!
현재 DB 시간: 2025-11-26T14:26:17.701Z
연결 종료

E:\mw_egps\conf>
```

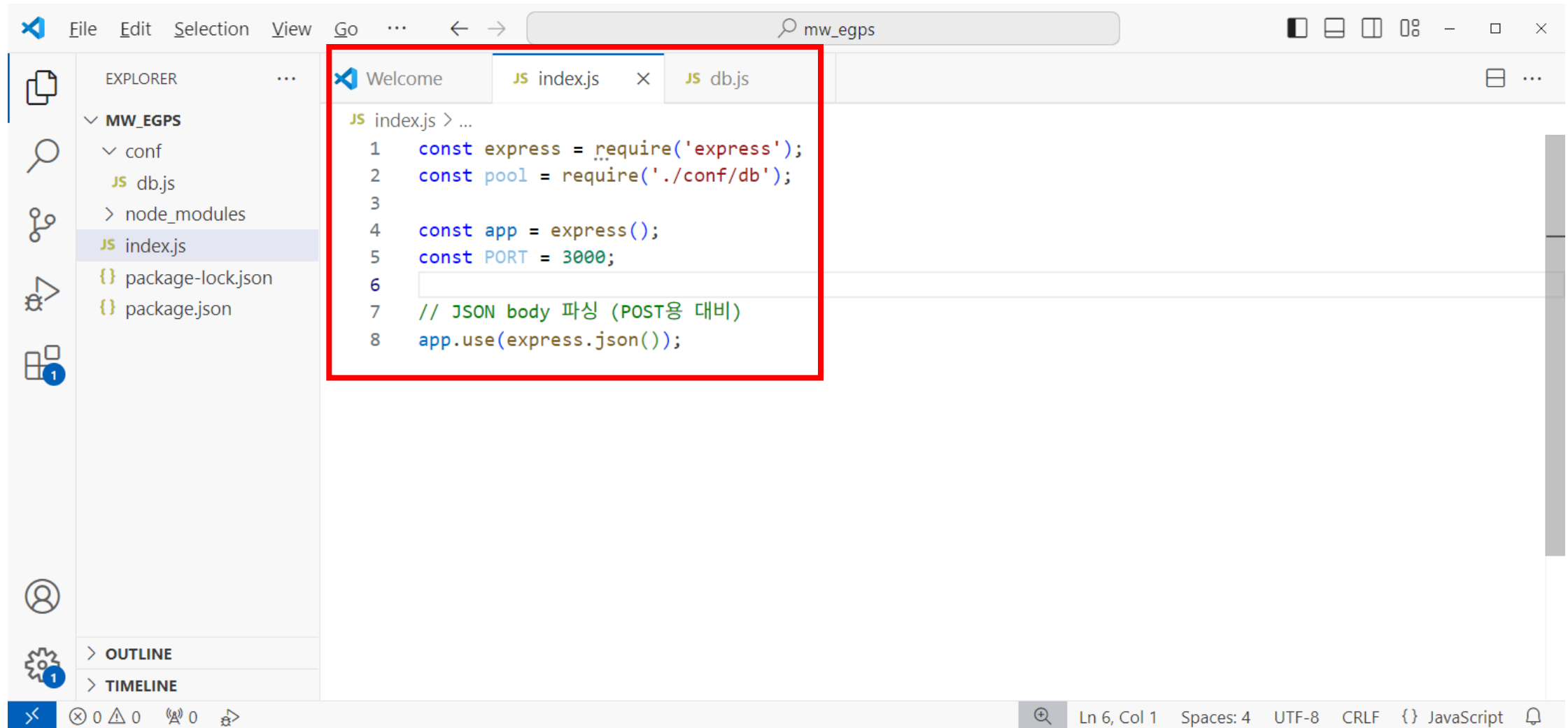
Ln 11, Col 1 Spaces: 4 UTF-8 CRLF {} JavaScript



The image shows a Visual Studio Code editor window with a search bar at the top containing 'mw_egps'. The file explorer on the left shows a project structure with 'Welcome', 'JS index.js', and 'JS db.js'. The 'JS db.js' file is open in the editor, and its content is highlighted with a red rectangle. The code is as follows:

```
conf > JS db.js > ...
1  const { Pool } = require('pg');
2
3  // PostgreSQL 설정을 본인 환경에 맞게 수정
4  const pool = new Pool({
5    host: 'localhost',      // 또는 DB 서버 IP
6    port: 5432,             // 기본 포트
7    user: 'postgres',       // DB 사용자 이름
8    password: 'apmsetup',   // DB 비밀번호
9    database: 'postgis_36_sample', // 사용할 DB 이름
10  });
11
12  // 다른 파일에서 사용할 수 있도록 export
13  module.exports = pool;
```

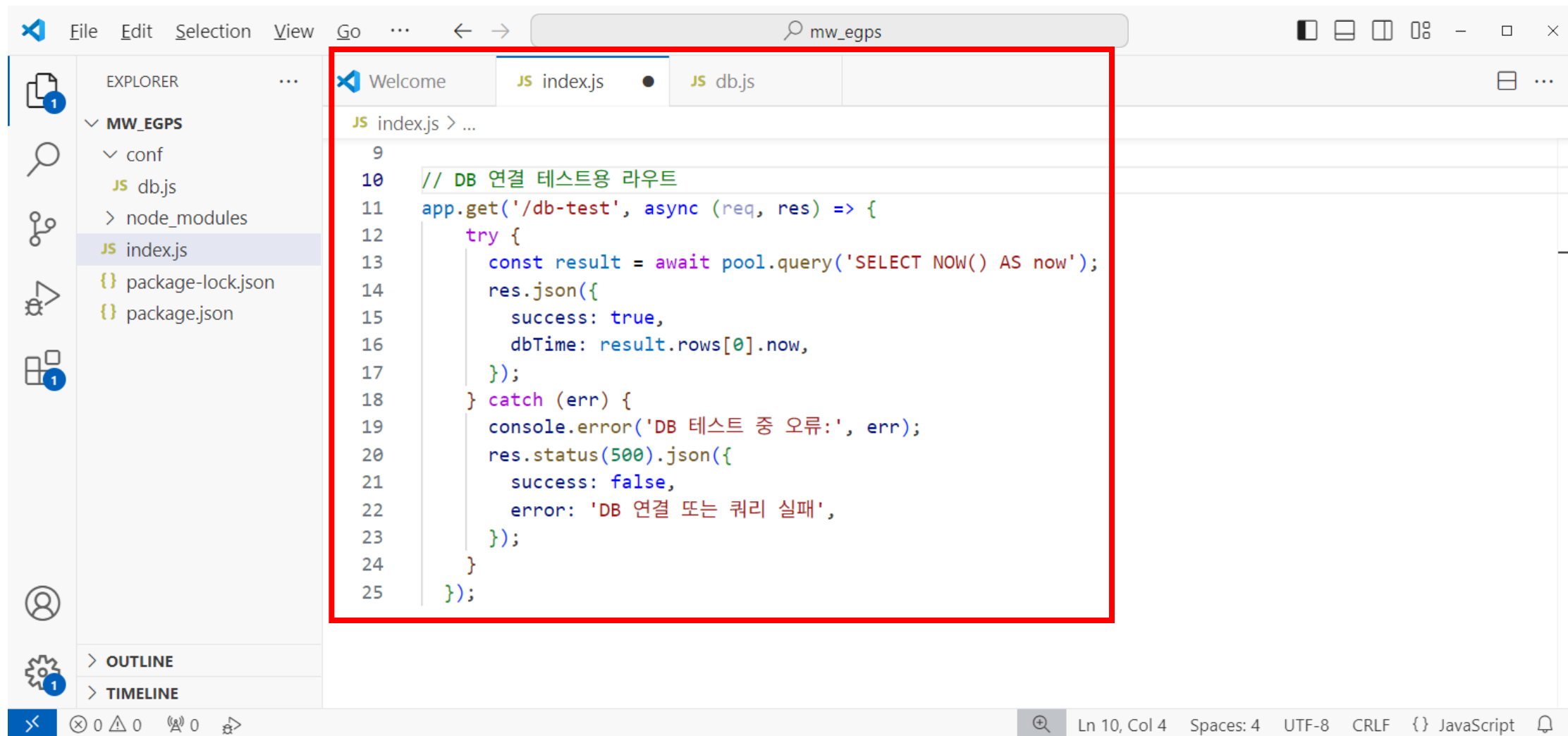
The status bar at the bottom indicates the current position is Line 3, Column 31, with 4 spaces, UTF-8 encoding, CRLF line endings, and the file is a JavaScript file.



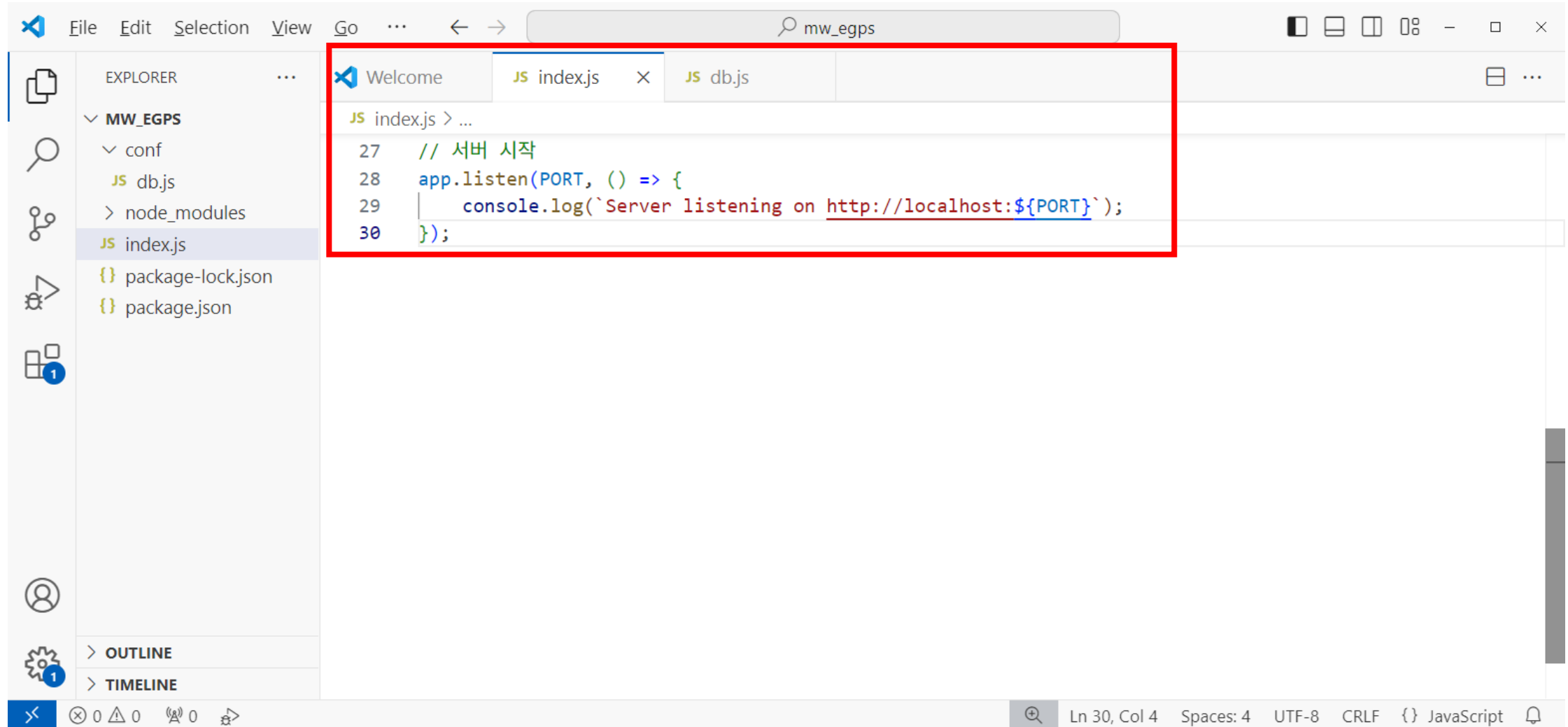
The screenshot shows the Visual Studio Code editor interface. The Explorer sidebar on the left displays the project structure for 'MW_EGPS', including a 'conf' directory with 'db.js', 'node_modules', 'index.js', 'package-lock.json', and 'package.json'. The 'index.js' file is selected and open in the editor. The code in 'index.js' is as follows:

```
JS index.js > ...
1  const express = require('express');
2  const pool = require('./conf/db');
3
4  const app = express();
5  const PORT = 3000;
6
7  // JSON body 파싱 (POST용 대비)
8  app.use(express.json());
```

The code is highlighted with a red rectangular box. The status bar at the bottom indicates the current position is Line 6, Column 1, with 4 spaces, UTF-8 encoding, CRLF line endings, and the file is a JavaScript document.



```
JS index.js > ...
9
10 // DB 연결 테스트용 라우트
11 app.get('/db-test', async (req, res) => {
12   try {
13     const result = await pool.query('SELECT NOW() AS now');
14     res.json({
15       success: true,
16       dbTime: result.rows[0].now,
17     });
18   } catch (err) {
19     console.error('DB 테스트 중 오류:', err);
20     res.status(500).json({
21       success: false,
22       error: 'DB 연결 또는 쿼리 실패',
23     });
24   }
25 });
```



```
File Edit Selection View Go ... < > mw_egps
Welcome JS index.js JS db.js
JS index.js > ...
27 // 서버 시작
28 app.listen(PORT, () => {
29   console.log('Server listening on http://localhost:${PORT}');
30 });
```

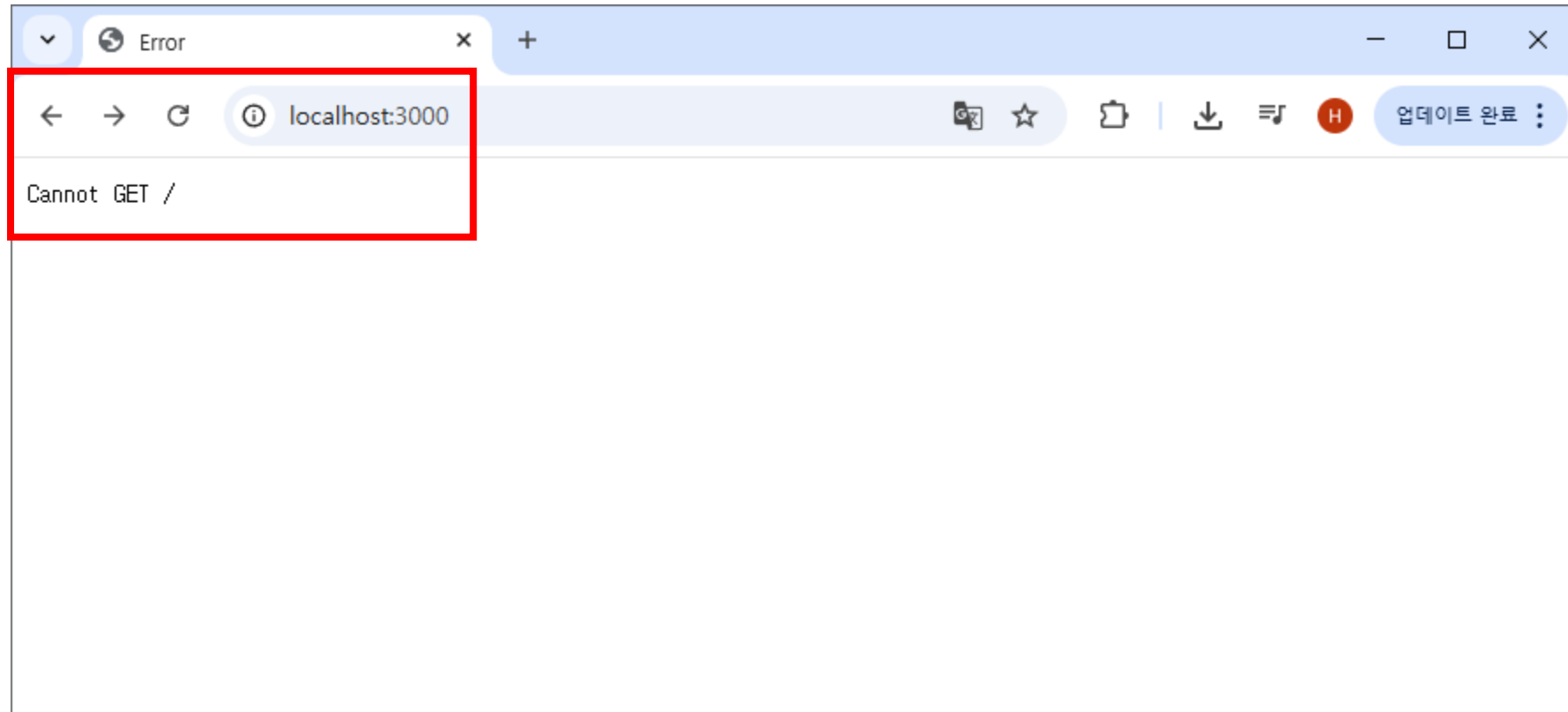
Ln 30, Col 4 Spaces: 4 UTF-8 CRLF {} JavaScript

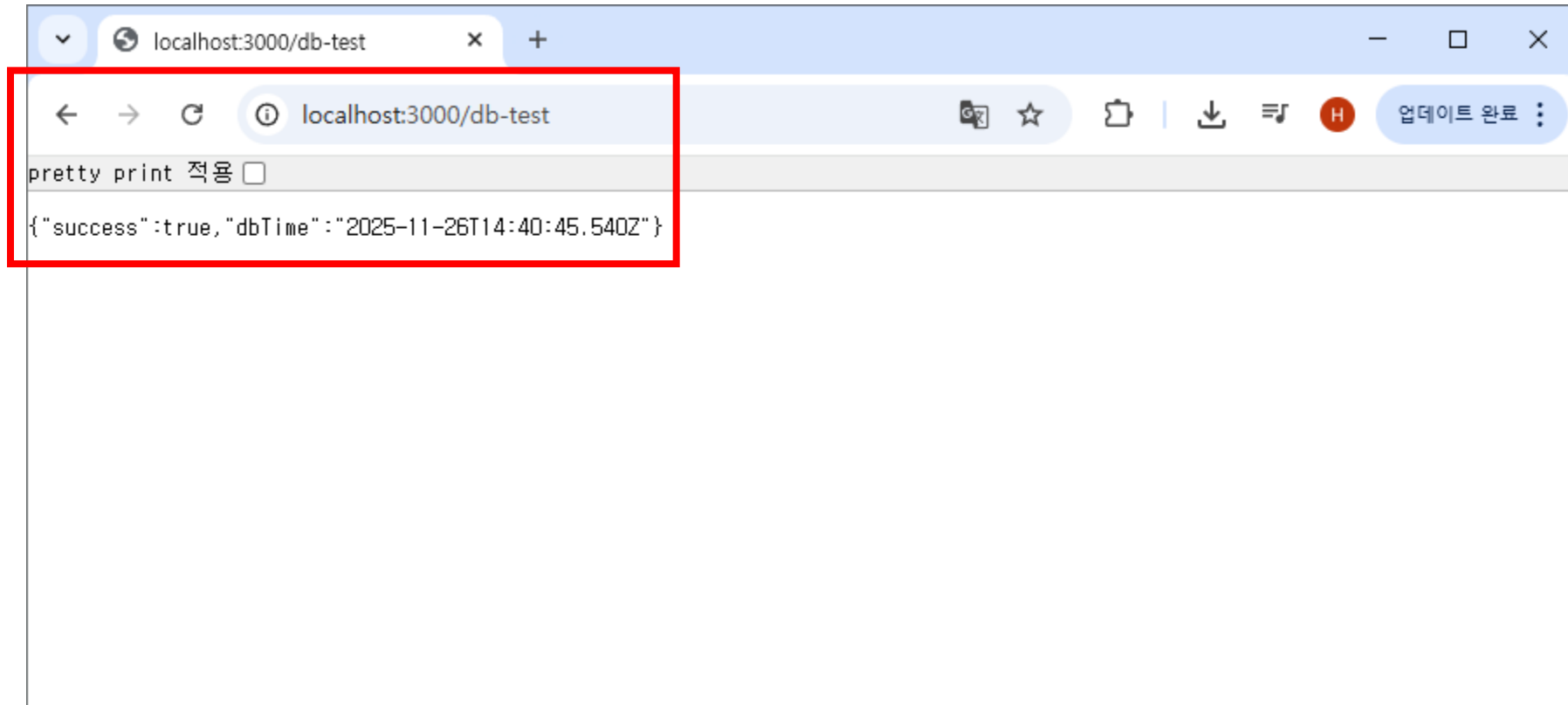


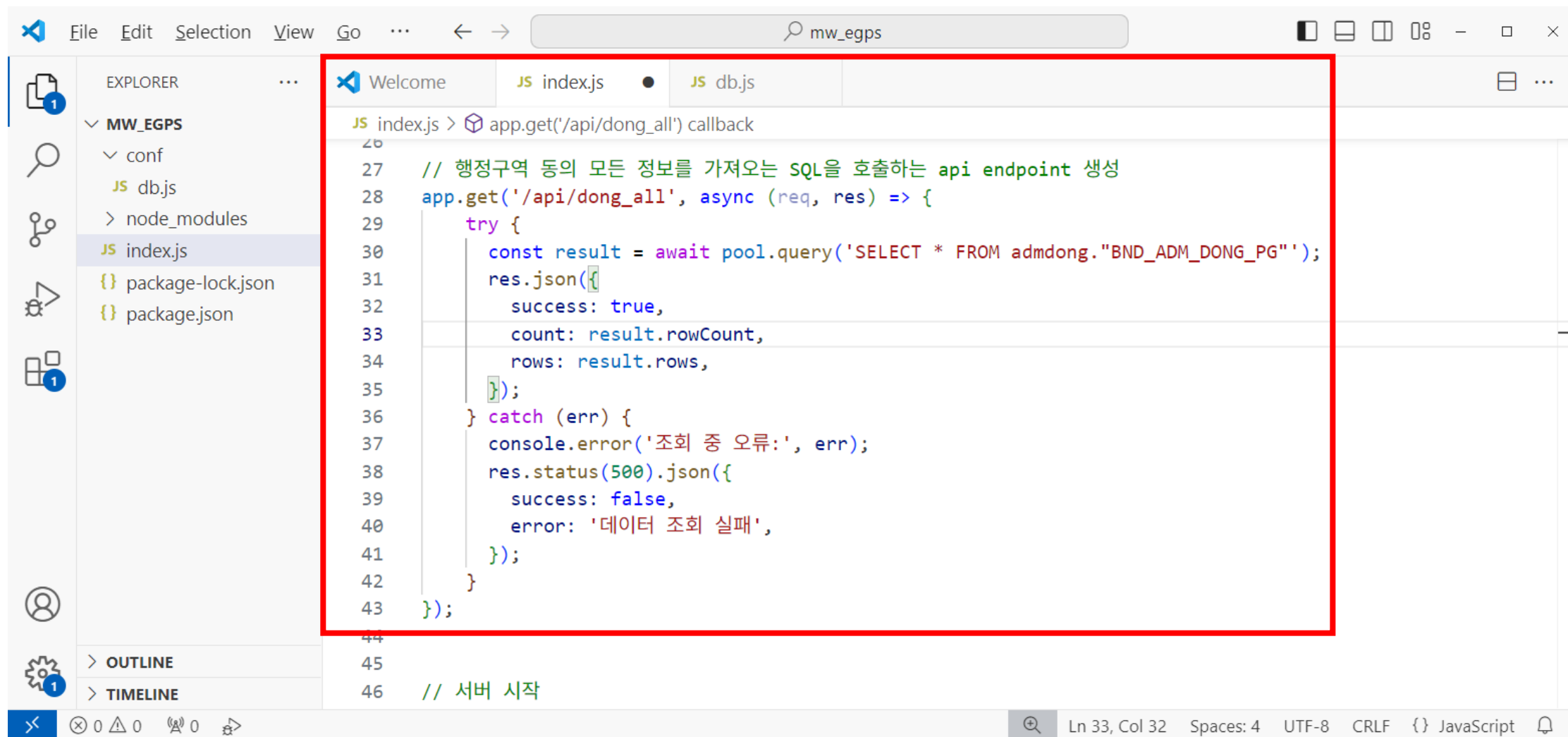
The screenshot shows the Visual Studio Code interface. The Explorer panel on the left displays the project structure for 'MW_EGPS', including 'conf', 'node_modules', 'index.js', 'package-lock.json', and 'package.json'. The main editor window shows the 'index.js' file with the following code:

```
27 // 서버 시작
28 app.listen(PORT, () => {
29   console.log(`Server listening on http://localhost:${PORT}`);
30 });
```

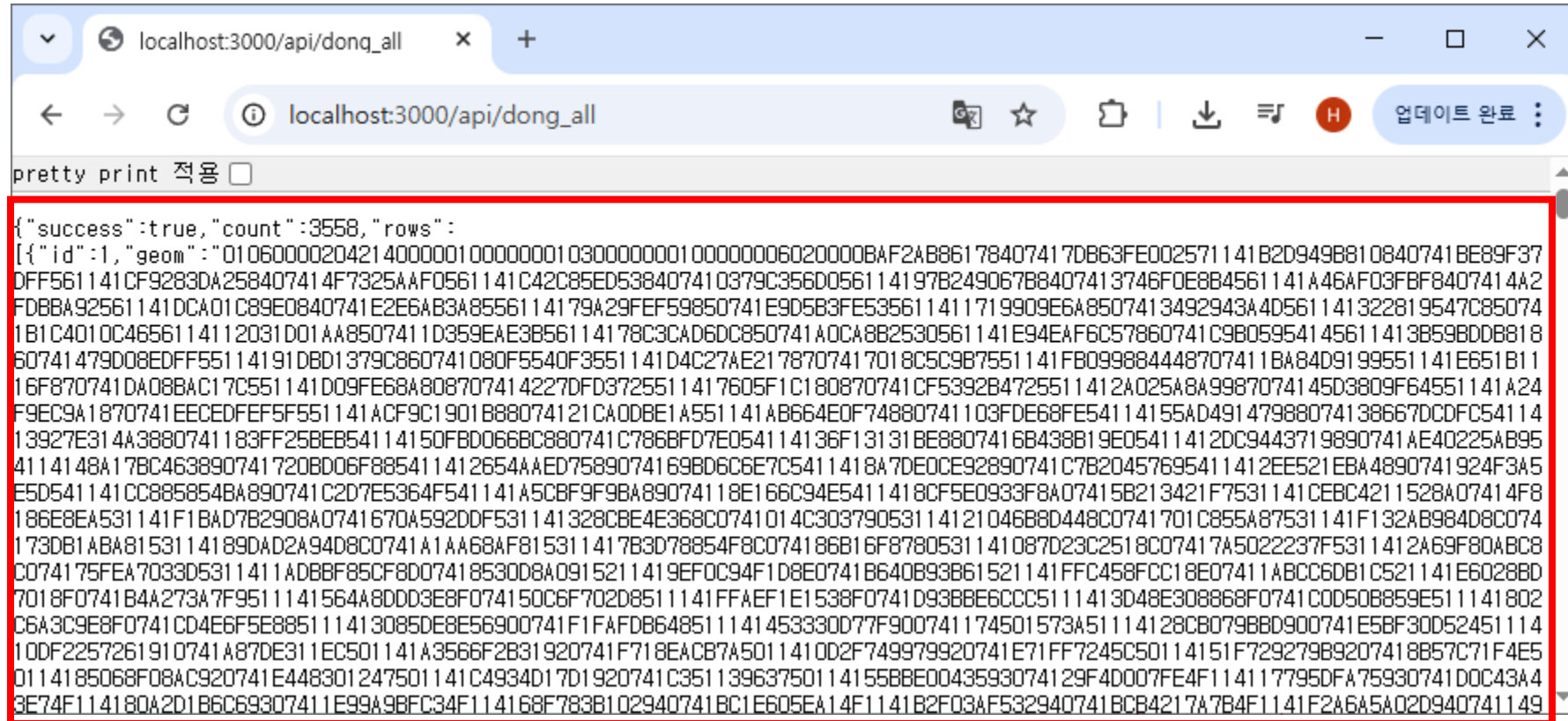
The terminal panel at the bottom shows the command 'node index' being executed, resulting in the output: 'Server listening on http://localhost:3000'.







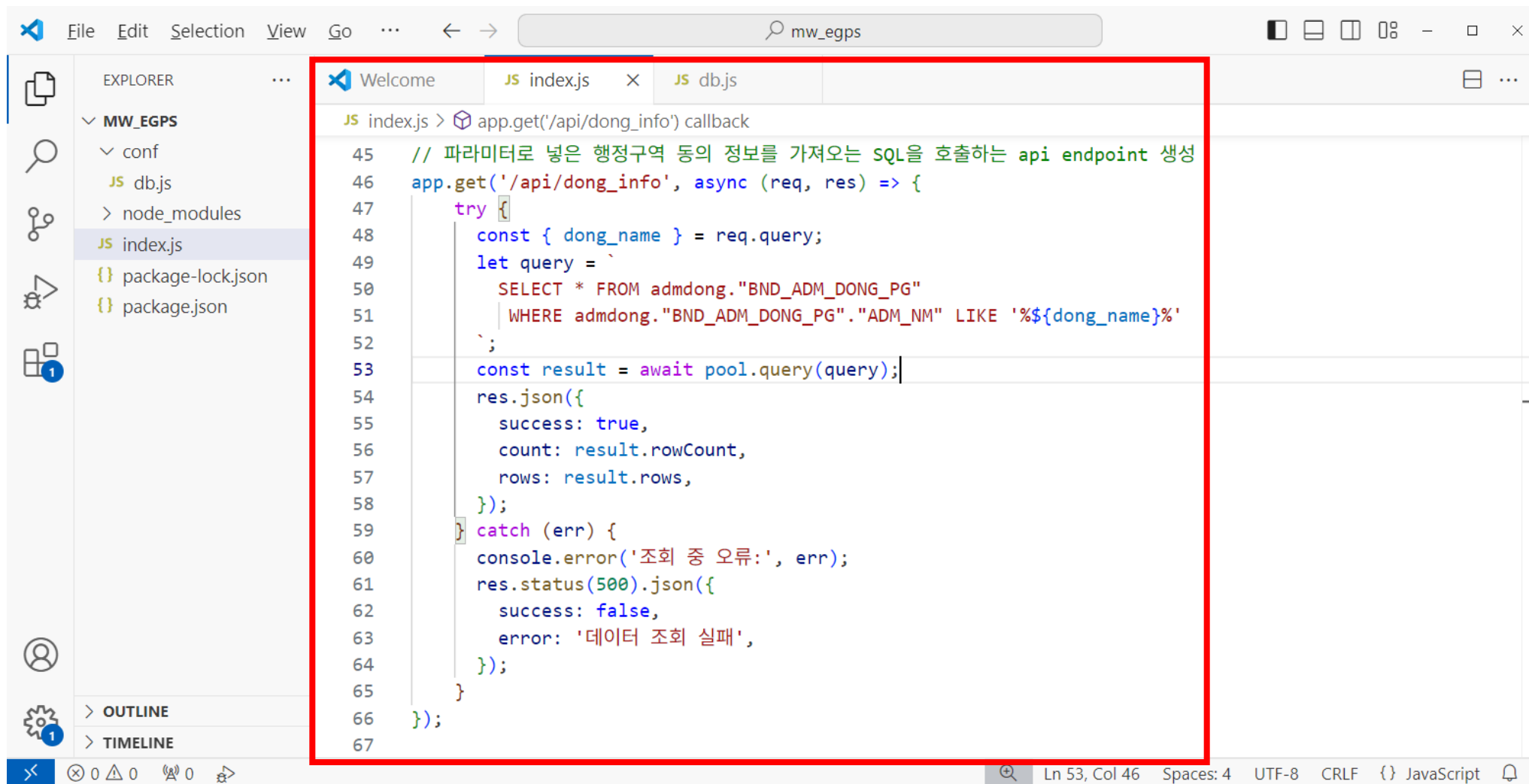
```
JS index.js > app.get('/api/dong_all') callback
26
27 // 행정구역 동의 모든 정보를 가져오는 SQL을 호출하는 api endpoint 생성
28 app.get('/api/dong_all', async (req, res) => {
29   try {
30     const result = await pool.query('SELECT * FROM admdong.\"BND_ADM_DONG_PG\"');
31     res.json({
32       success: true,
33       count: result.rowCount,
34       rows: result.rows,
35     });
36   } catch (err) {
37     console.error('조회 중 오류:', err);
38     res.status(500).json({
39       success: false,
40       error: '데이터 조회 실패',
41     });
42   }
43 });
44
45
46 // 서버 시작
```



```
{
  "success": true,
  "count": 3558,
  "rows": [
    {
      "id": 1,
      "geom": "0106000020421400000100000001030000000100000006020000BAF2AB86178407417DB63FE002571141B2D949B810840741BE89F37DFF561141CF9283DA258407414F7325AAF0561141C42C85ED538407410379C356D056114197B249067B8407413746F0E8B4561141A46AF03FBF8407414A2FDBBA92561141DCA01C89E0840741E2E6AB3A8556114179A29FEF59850741E9D5B3FE535611411719909E6A8507413492943A4D561141322819547C850741B1C4010C4656114112031D01AA8507411D359EAE3B56114178C3CAD6DC850741A0CA8B2530561141E94EAF6C57860741C9B05954145611413B59BDD8B1860741479D08EDFF55114191DBD1379C860741080F5540F3551141D4C27AE2178707417018C5C9B7551141FB099884448707411BA84D9199551141E651B1116F870741DA08BAC17C551141D09FE68A808707414227DFD3725511417605F1C180870741CF5392B4725511412A025A8A9987074145D3809F64551141A24F9EC9A1870741EECEDFEF5F551141ACF9C1901B88074121CA0DBE1A551141AB664E0F74880741103FDE68FE54114155AD49147988074138667DCDFC5411413927E314A3880741183FF258EB54114150FBD066BC880741C786BFD7E054114136F13131BE8807416B438B19E05411412DC9443719890741AE40225AB954114148A17BC463890741720BD06F885411412654AAED7589074169BD6C6E7C5411418A7DE0CE92890741C7B2D457695411412EE521EBA4890741924F3A5E5D541141CC885854BA890741C2D7E5364F541141A5CBF9F9BA89074118E166C94E5411418CF5E0933F8A07415B213421F7531141CEBC4211528A07414F8186E8EA531141F1BAD7B2908A0741670A592DDF531141328CBE4E368C0741014C30379053114121046B8D448C0741701C855A87531141F132AB984D8C074173DB1ABA8153114189DAD2A94D8C0741A1AA68AF815311417B3D78854F8C074186B16F8780531141087D23C2518C07417A5022237F5311412A69F80ABC8C074175FEA7033D5311411ADB8F85CF8D07418530D8A0915211419EF0C94F1D8E0741B640B93861521141FFC458FCC18E07411ABCC6DB1C521141E6028BD7018F0741B4A273A7F951141564A8DD03E8F074150C6F702D851141FFAEF1E1538F0741D938BE6CC511413D48E308868F0741C0D50B859E51141802C6A3C9E8F0741CD4E6F5E88511413085DE8E56900741F1FAFDB6485114145333D077F900741174501573A5114128C80798BD900741E58F30D5245114110DF2257261910741A87DE311EC501141A3566F2B31920741F718EACB7A5011410D2F749979920741E71FF7245C50114151F729279B9207418B57C71F4E50114185068F08AC920741E448301247501141C4934D17D1920741C351139637501141558BE0043593074129F4D007FE4F114117795DFA75930741D0C43A43E74F114180A2D1B6C69307411E99A98FC34F114168F783B102940741BC1E605EA14F1141B2F03AF532940741BCB4217A7B4F1141F2A6A5A02D940741149
```

1. 공간정보 프로그래밍

Node.js – PostgreSQL 연동



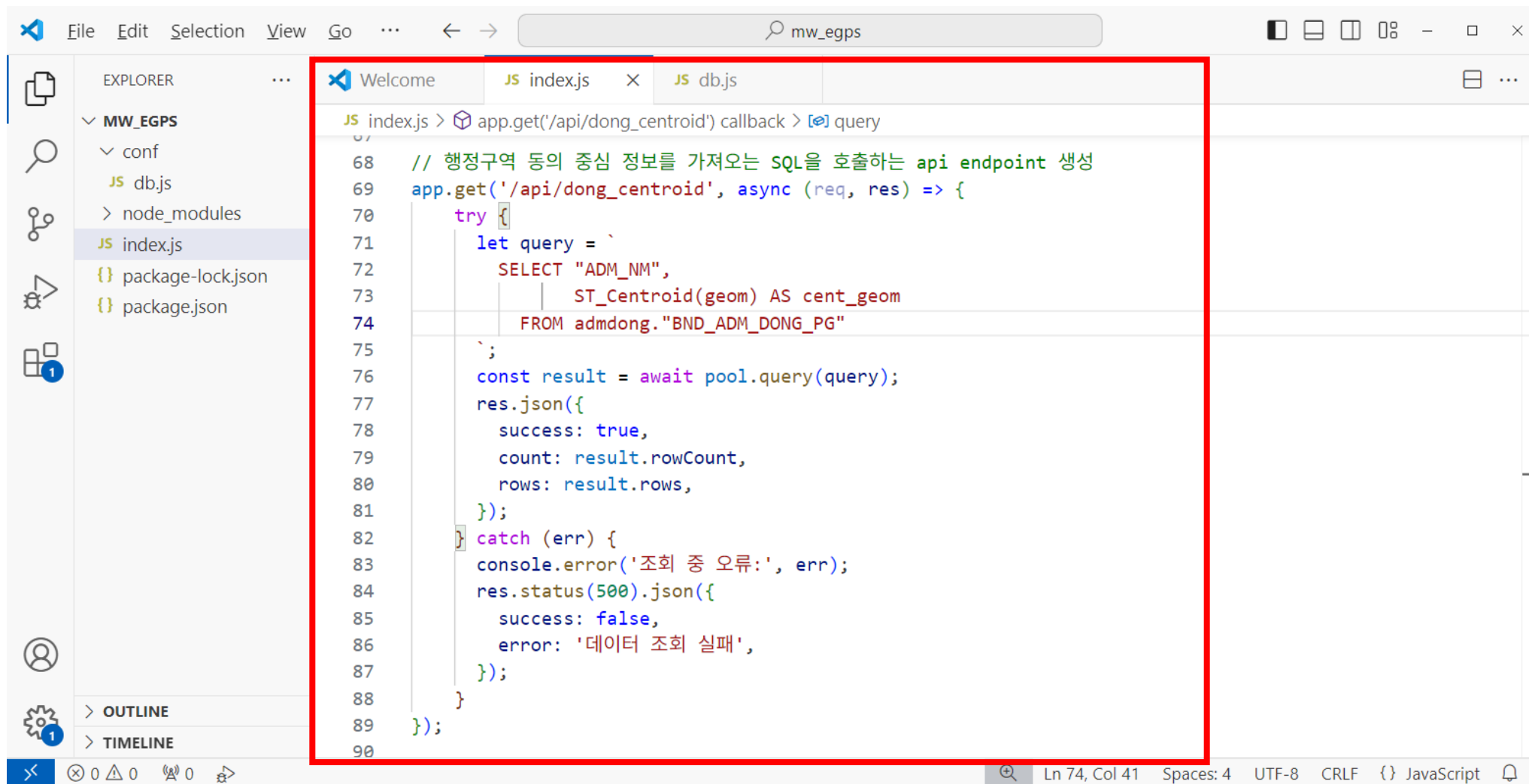
```
JS index.js > app.get('/api/dong_info') callback
45 // 파라미터로 넣은 행정구역 동의 정보를 가져오는 SQL을 호출하는 api endpoint 생성
46 app.get('/api/dong_info', async (req, res) => {
47   try {
48     const { dong_name } = req.query;
49     let query = `
50       SELECT * FROM admdong."BND_ADM_DONG_PG"
51       WHERE admdong."BND_ADM_DONG_PG"."ADM_NM" LIKE '%${dong_name}%'
52     `;
53     const result = await pool.query(query);
54     res.json({
55       success: true,
56       count: result.rowCount,
57       rows: result.rows,
58     });
59   } catch (err) {
60     console.error('조회 중 오류:', err);
61     res.status(500).json({
62       success: false,
63       error: '데이터 조회 실패',
64     });
65   }
66 });
67
```



```
localhost:3000/api/dong_all
localhost:3000/api/dong_all
pretty print 적용
{"success":true,"count":3558,"rows":
[{"id":1,"geom":"0106000020421400000100000001030000000100000006020000BAF2AB86178407417DB63FE002571141B2D949B810840741BE89F37
DFF561141CF9283DA258407414F7325AAF0561141C42C85ED538407410379C356D056114197B249067B8407413746F0E8B4561141A46AF03FBF8407414A2
FDBBA92561141DCA01C89E0840741E2E6AB3A8556114179A29FEF59850741E9D5B3FE535611411719909E6A8507413492943A4D561141322819547C85074
1B1C4010C4656114112031D01AA8507411D359EAE3B56114178C3CAD6DC850741A0CA8B2530561141E94EAF6C57860741C9B05954145611413B59BDD8B18
60741479D08EDFF55114191DBD1379C860741080F5540F3551141D4C27AE2178707417018C5C9B7551141FB099884448707411BA84D9199551141E651B11
16F870741DA08BAC17C551141D09FE68A808707414227DFD3725511417605F1C180870741CF5392B4725511412A025A8A9987074145D3809F64551141A24
F9EC9A1870741EECEDFEF5F551141ACF9C1901B88074121CA0DBE1A551141AB664E0F74880741103FDE68FE54114155AD49147988074138667DCDFC54114
13927E314A3880741183FF258EB54114150FBD066BC880741C786BFD7E054114136F13131BE8807416B438B19E05411412DC9443719890741AE40225AB95
4114148A17BC463890741720BD06F885411412654AAED7589074169BD6C6E7C5411418A7DE0CE92890741C7B2D457695411412EE521EBA4890741924F3A5
E5D541141CC885854BA890741C2D7E5364F541141A5CBF9F9BA89074118E166C94E5411418CF5E0933F8A07415B213421F7531141CEBC4211528A07414F8
186E8EA531141F1BAD7B2908A0741670A592DDF531141328CBE4E368C0741014C30379053114121046B8D448C0741701C855A87531141F132AB984D8C074
173DB1ABA8153114189DAD2A94D8C0741A1AA68AF815311417B3D78854F8C074186B16F8780531141087D23C2518C07417A5022237F5311412A69F80ABC8
C074175FEA7033D5311411ADB8F85CF8D07418530D8A0915211419EFOC94F1D8E0741B640B93861521141FFC458FCC18E07411ABCC6DB1C521141E6028BD
7018F0741B4A273A7F9511141564A8DD03E8F074150C6F702D8511141FFAEF1E1538F0741D938BE6CC5111413D48E308868F0741C0D50B859E511141802
C6A3C9E8F0741CD4E6F5E885111413085DE8E56900741F1FAFDB64851114145333D077F900741174501573A51114128C80798BD900741E58F30D52451114
10DF2257261910741A87DE311EC501141A3566F2B31920741F718EACB7A5011410D2F749979920741E71FF7245C50114151F729279B9207418B57C71F4E5
0114185068F08AC920741E448301247501141C4934D17D1920741C351139637501141558BE0043593074129F4D007FE4F114117795DFA75930741D0C43A4
3E74F114180A2D1B6C69307411E99A98FC34F114168F783B102940741BC1E605EA14F1141B2F03AF532940741BCB4217A7B4F1141F2A6A5A02D940741149
```

1. 공간정보 프로그래밍

Node.js – PostgreSQL 연동



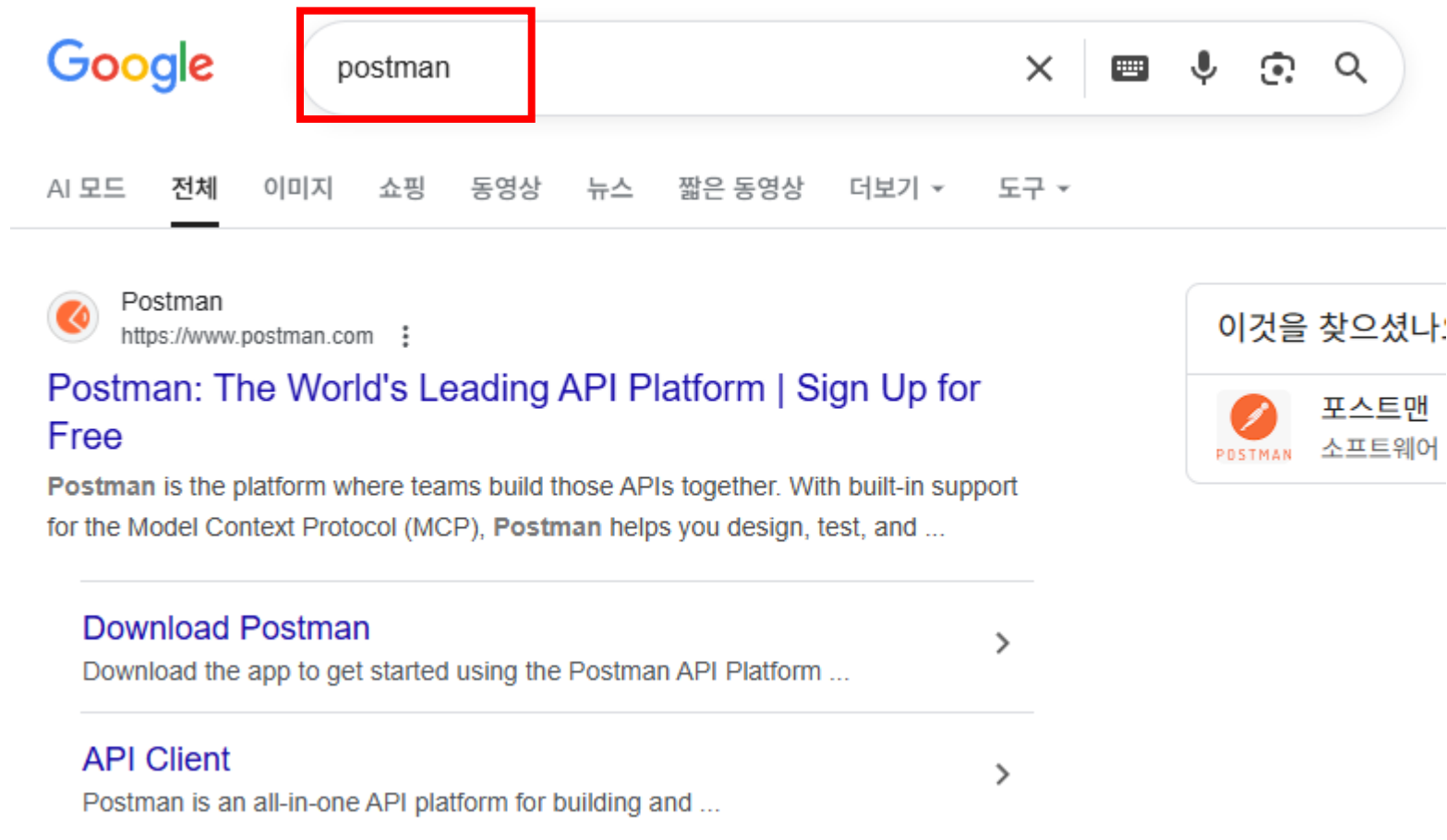
```
JS index.js > app.get('/api/dong_centroid') callback > query
68 // 행정구역 동의 중심 정보를 가져오는 SQL을 호출하는 api endpoint 생성
69 app.get('/api/dong_centroid', async (req, res) => {
70   try {
71     let query = `
72       SELECT "ADM_NM",
73             ST_Centroid(geom) AS cent_geom
74     FROM admdong."BND_ADM_DONG_PG"
75   `;
76     const result = await pool.query(query);
77     res.json({
78       success: true,
79       count: result.rowCount,
80       rows: result.rows,
81     });
82   } catch (err) {
83     console.error('조회 중 오류:', err);
84     res.status(500).json({
85       success: false,
86       error: '데이터 조회 실패',
87     });
88   }
89 });
90
```

```
localhost:3000/api/dong_centroid
localhost:3000/api/dong_centroid
pretty print 적용
{"success":true,"count":3558,"rows":[{"ADM_NM":"충장동","cent_geom":"0101000020421400005F0E27C94879074134CA81D75F521141"},
{"ADM_NM":"동명동","cent_geom":"010100002042140000BAF324B4AD950741BFE24CD171521141"}, {"ADM_NM":"안좌
모", "cent_geom":"010100002042140000141A09A50B53FD40004C01538A050D41"}, {"ADM_NM":"팔금
모", "cent_geom":"010100002042140000695F2B09F89BFD400D700BCE8EBD0D41"}, {"ADM_NM":"암태
모", "cent_geom":"010100002042140000A83F78EDD7EDFC407D78CE37C8800E41"}, {"ADM_NM":"내동
모", "cent_geom":"010100002042140000EB6401382DFF1141DC210B51224E1141"}, {"ADM_NM":"계림1
모", "cent_geom":"01010000204214000044DEE4F7187F07410E8599C697611141"}, {"ADM_NM":"삼동
모", "cent_geom":"01010000204214000076E854D97EE711413727B16EE8E30D41"}, {"ADM_NM":"미조
모", "cent_geom":"010100002042140000B7CE86CD4D00124199912322B9000D41"}, {"ADM_NM":"학운
모", "cent_geom":"0101000020421400003C95367D0B01084156BE5AAC4A2C1141"}, {"ADM_NM":"남하
모", "cent_geom":"0101000020421400004180BCFAF5961141AA8D0BC841E41441"}, {"ADM_NM":"신원
모", "cent_geom":"010100002042140000C83B7F6478571141A2F879F653271441"}, {"ADM_NM":"가조
모", "cent_geom":"0101000020421400001C3DFF4D58E41141994D8D9CB41B1541"}, {"ADM_NM":"가북
모", "cent_geom":"0101000020421400006FD651D89DC411418A3DBCECF9CB51541"}, {"ADM_NM":"계림2
모", "cent_geom":"010100002042140000A13454BD3D91074144D1F58D23631141"}, {"ADM_NM":"산청
모", "cent_geom":"0101000020421400007D5BCBFF05331141352E42F37C151341"}, {"ADM_NM":"차황
모", "cent_geom":"010100002042140000A026739C52701141E736DA32F1851341"}, {"ADM_NM":"오부
모", "cent_geom":"010100002042140000453E4800012611415146AE045CA61341"}, {"ADM_NM":"생초
모", "cent_geom":"010100002042140000B84DA00920E71041F5BB6CAC42C51341"}, {"ADM_NM":"노형
```

전자항법과 GPS

1. 공간정보 프로그래밍

1-3. Etc. Tools



A screenshot of a Google search for 'postman'. The search bar contains the word 'postman' and is highlighted with a red rectangle. Below the search bar, the search results for 'Postman' are displayed. The main result is 'Postman: The World's Leading API Platform | Sign Up for Free' with the URL 'https://www.postman.com'. Below this, there are two links: 'Download Postman' and 'API Client'. To the right of the search results, there is a sidebar with the text '이것을 찾으셨나:' and a logo for '포스트맨 소프트웨어' (Postman Software).

Google postman

AI 모드 전체 이미지 쇼핑 동영상 뉴스 짧은 동영상 더보기 도구

 Postman
https://www.postman.com

Postman: The World's Leading API Platform | Sign Up for Free

Postman is the platform where teams build those APIs together. With built-in support for the Model Context Protocol (MCP), Postman helps you design, test, and ...

Download Postman >
Download the app to get started using the Postman API Platform ...

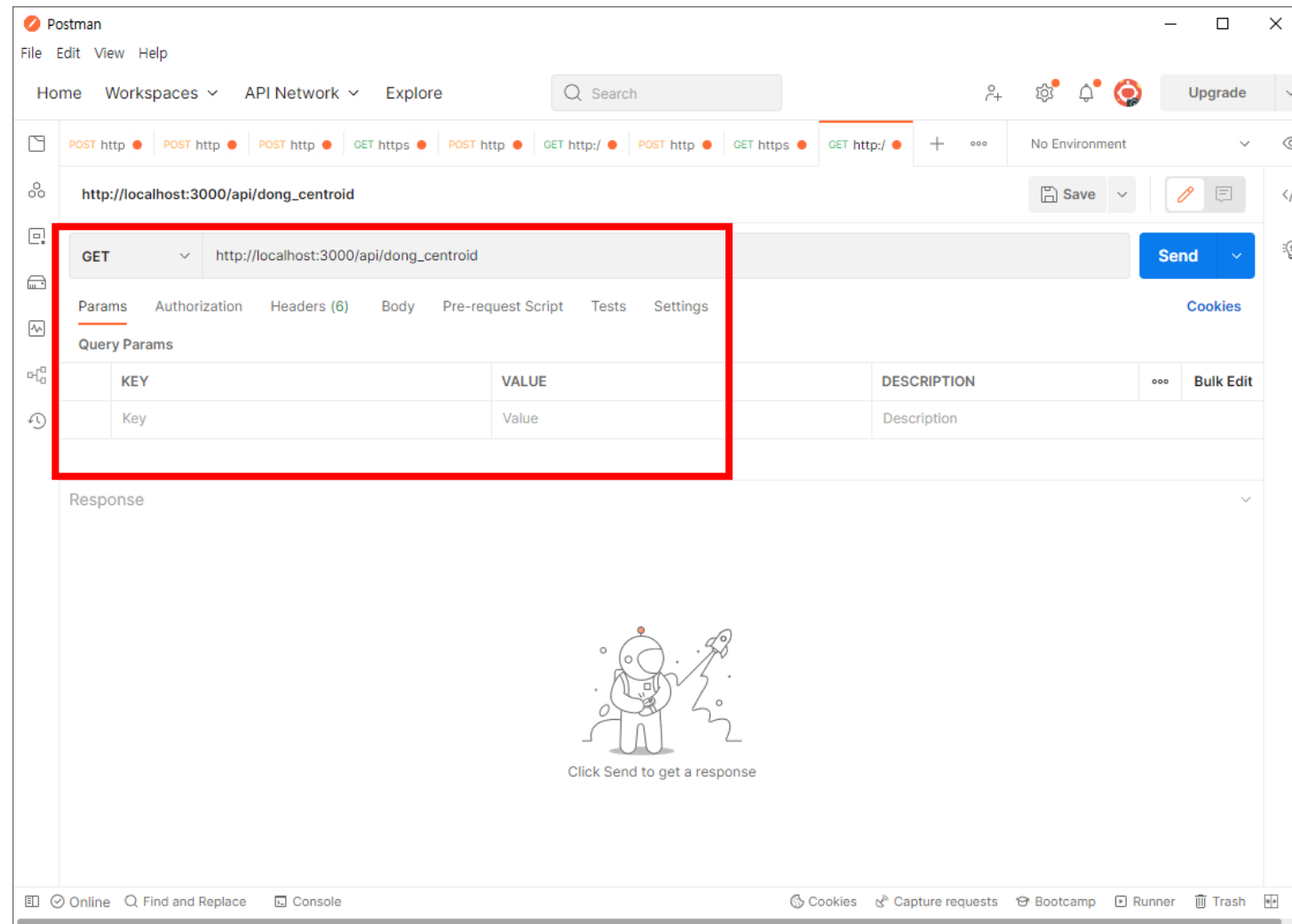
API Client >
Postman is an all-in-one API platform for building and ...

이것을 찾으셨나:

 포스트맨
소프트웨어

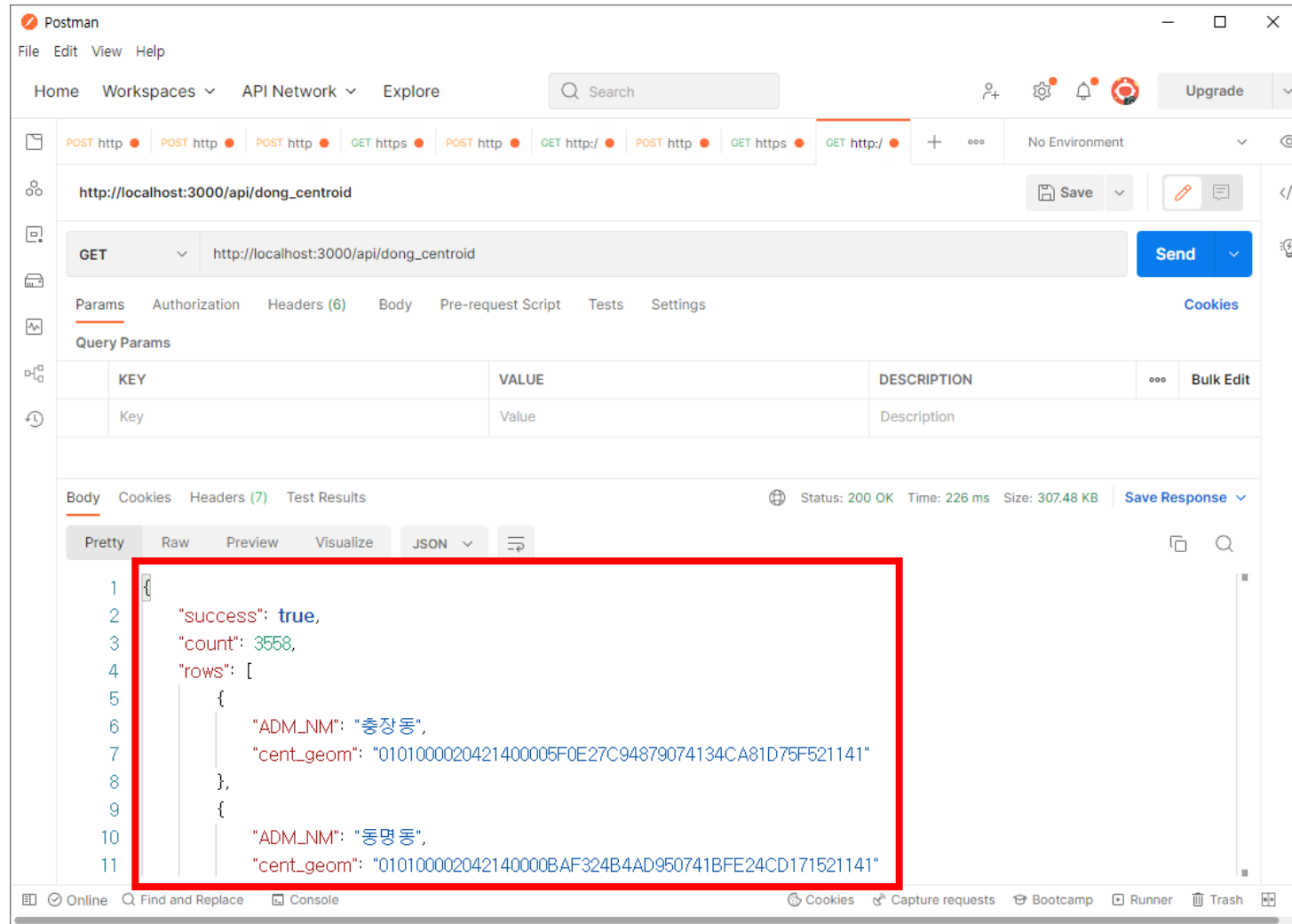
1. 공간정보 프로그래밍

Postman



1. 공간정보 프로그래밍

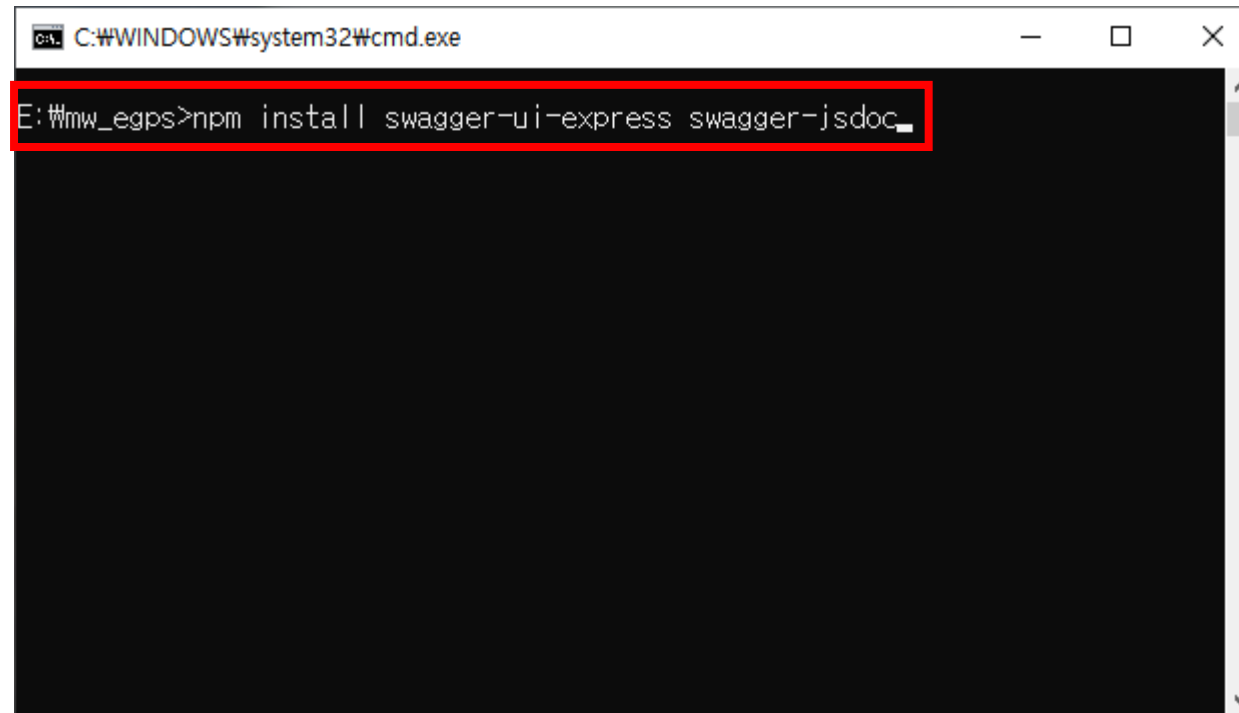
Postman



The image shows the Postman application window. At the top, there's a menu bar (File, Edit, View, Help) and a toolbar with icons for Home, Workspaces, API Network, and Explore. Below this is a search bar and an 'Upgrade' button. The main area displays a GET request to `http://localhost:3000/api/dong_centroid`. The 'Params' tab is selected, showing a table with columns KEY, VALUE, and DESCRIPTION. The 'Body' tab is also visible, showing the response in JSON format. The response is a JSON object with the following structure:

```
{
  "success": true,
  "count": 3558,
  "rows": [
    {
      "ADM_NM": "충장동",
      "cent_geom": "0101000020421400005F0E27C94879074134CA81D75F521141"
    },
    {
      "ADM_NM": "동명동",
      "cent_geom": "010100002042140000BAF324B4AD950741BFE24CD171521141"
    }
  ]
}
```

The response status is 200 OK, with a time of 226 ms and a size of 307.48 KB. The 'Save Response' button is visible. The bottom status bar includes links to Online, Find and Replace, Console, Cookies, Capture requests, Bootcamp, Runner, and Trash.



```
C:\WINDOWS\system32\cmd.exe
E:\mw_egps>npm install swagger-ui-express swagger-jsdoc
```



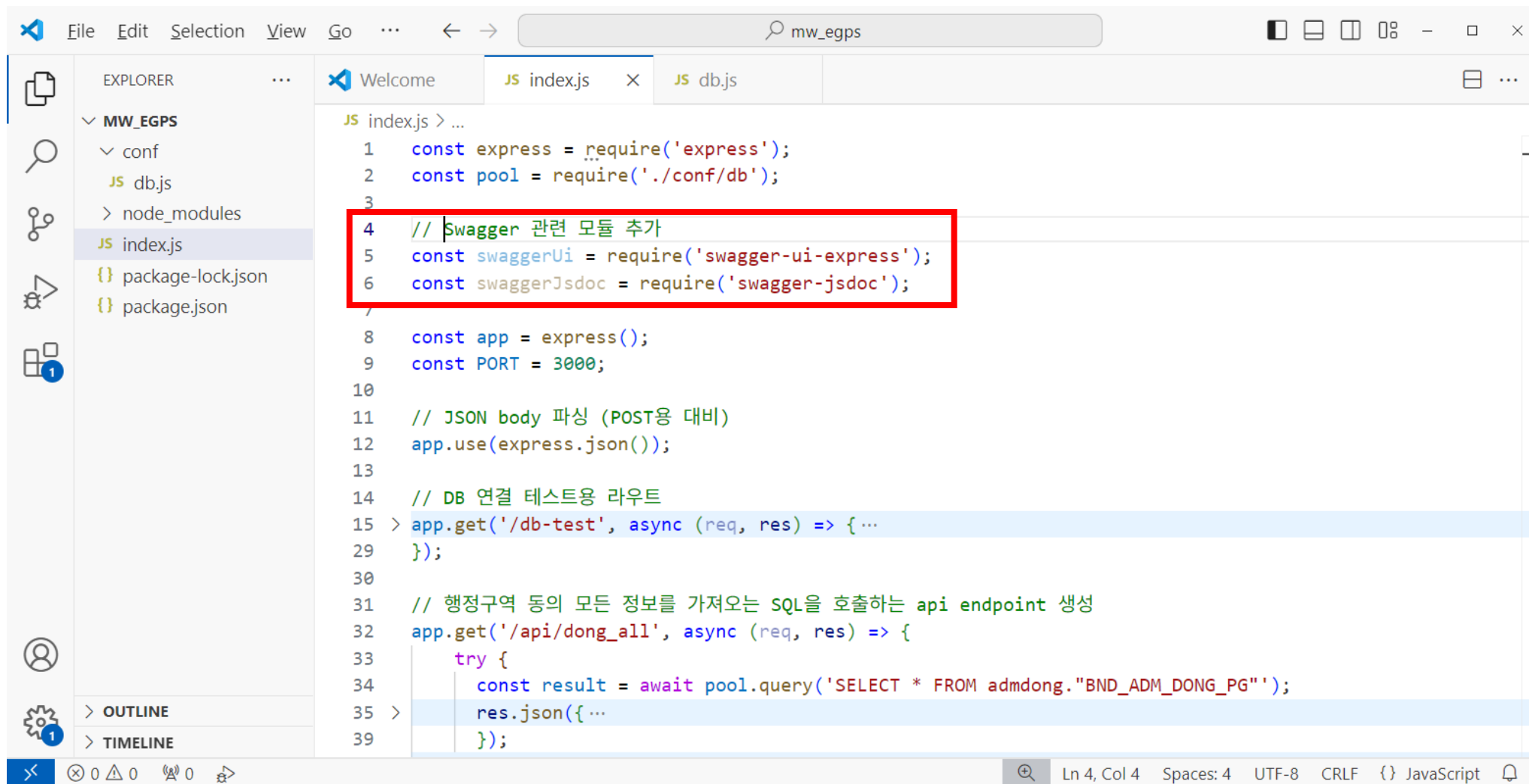
```
C:\WINDOWS\system32\cmd.exe
npm WARN EBADENGINE }
npm WARN deprecated inflight@1.0.6: This module is not supported, and leaks
memory. Do not use it. Check out lru-cache if you want a good and tested w
ay to coalesce async requests by a key value, which is much more comprehens
ive and powerful.
npm WARN deprecated lodash.isequal@4.5.0: This package is deprecated. Use r
equire('node:util').isDeepStrictEqual instead.
npm WARN deprecated lodash.get@4.4.2: This package is deprecated. Use the o
ptional chaining (?.) operator instead.
npm WARN deprecated glob@7.1.6: Glob versions prior to v9 are no longer sup
ported

added 33 packages, and audited 113 packages in 15s

20 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

E:\Wmw_egps>
```



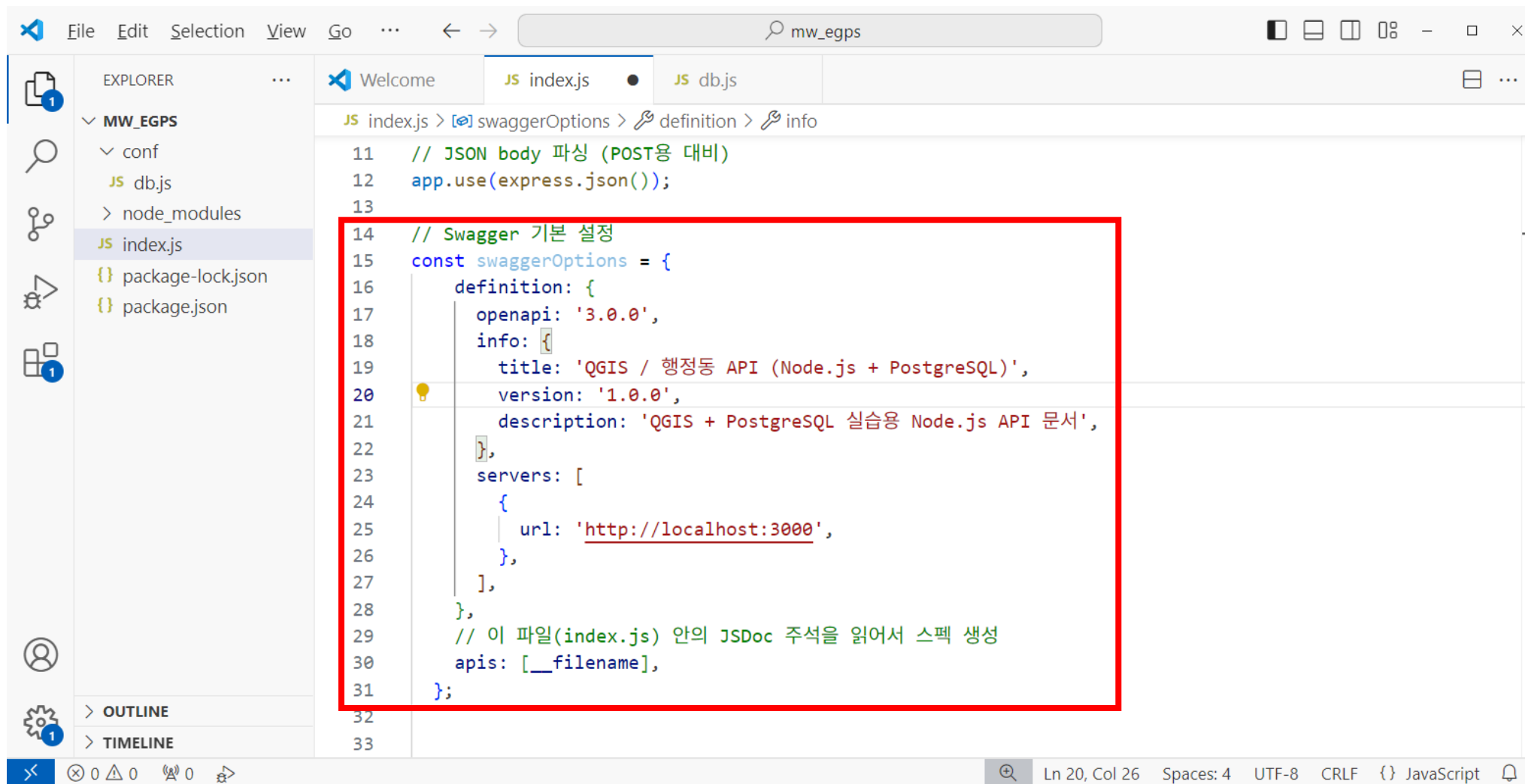
The screenshot shows the Visual Studio Code editor with the file explorer on the left and the code editor in the center. The file explorer shows the project structure for 'MW_EGPS', including 'conf', 'node_modules', and 'index.js'. The code editor displays the contents of 'index.js', with lines 4, 5, and 6 highlighted in a red box. These lines are:

```
4 // Swagger 관련 모듈 추가
5 const swaggerUi = require('swagger-ui-express');
6 const swaggerJsdoc = require('swagger-jsdoc');
```

The rest of the code in the editor includes:

```
1 const express = require('express');
2 const pool = require('./conf/db');
3
7
8 const app = express();
9 const PORT = 3000;
10
11 // JSON body 파싱 (POST용 대비)
12 app.use(express.json());
13
14 // DB 연결 테스트용 라우트
15 app.get('/db-test', async (req, res) => { ...
29 });
30
31 // 행정구역 동의 모든 정보를 가져오는 SQL을 호출하는 api endpoint 생성
32 app.get('/api/dong_all', async (req, res) => {
33   try {
34     const result = await pool.query('SELECT * FROM admdong."BND_ADM_DONG_PG"');
35     res.json({ ...
39   });
```

The status bar at the bottom indicates the current line and column: 'Ln 4, Col 4'. Other status bar information includes 'Spaces: 4', 'UTF-8', 'CRLF', and 'JavaScript'.

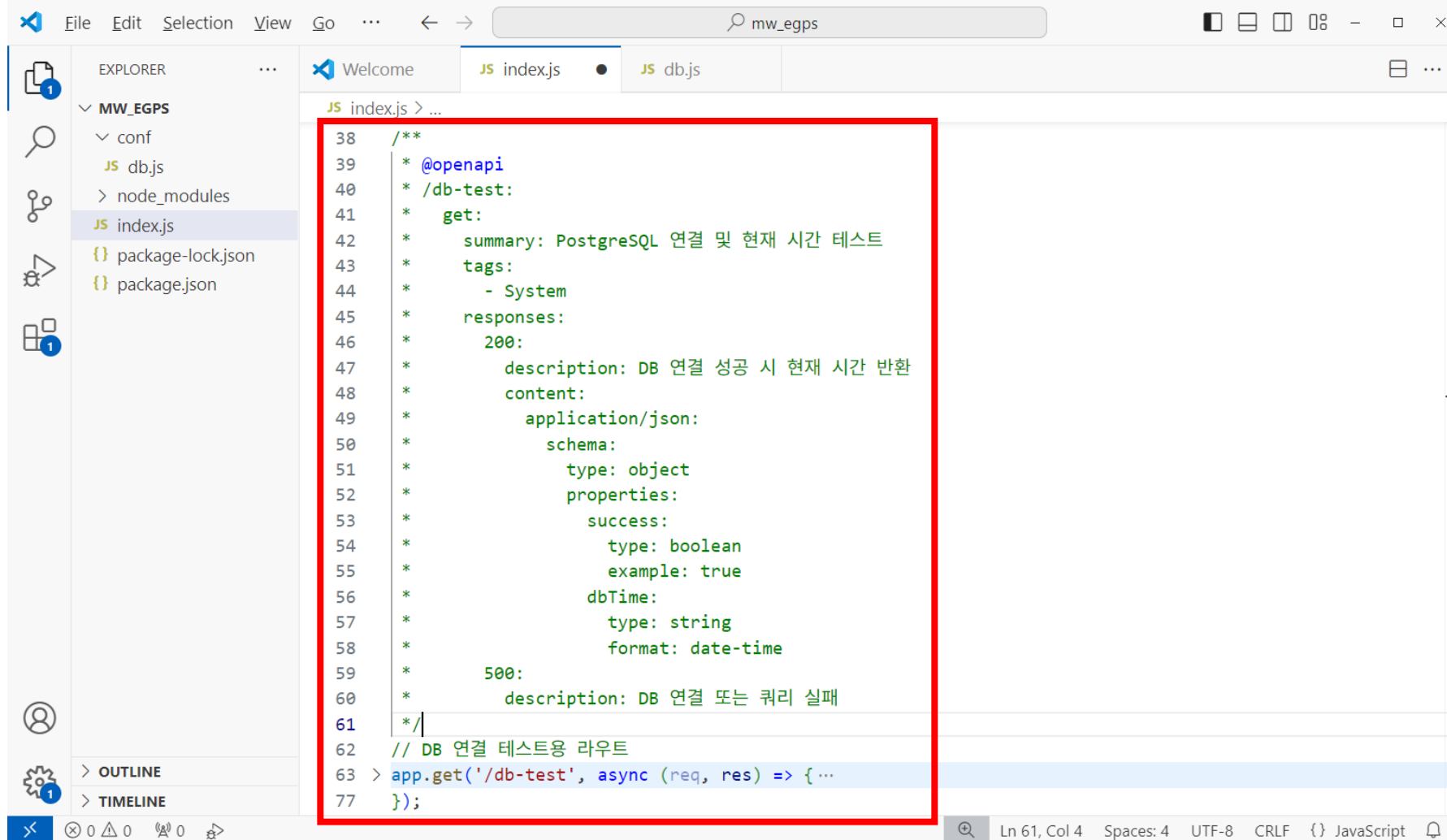


```
File Edit Selection View Go ... < > mw_egps
EXPLORER
  MW_EGPS
    conf
      db.js
    node_modules
    index.js
    package-lock.json
    package.json
  OUTLINE
  TIMELINE

JS index.js > swaggerOptions > definition > info
11 // JSON body 파싱 (POST용 대비)
12 app.use(express.json());
13
14 // Swagger 기본 설정
15 const swaggerOptions = {
16   definition: {
17     openapi: '3.0.0',
18     info: {
19       title: 'QGIS / 행정동 API (Node.js + PostgreSQL)',
20       version: '1.0.0',
21       description: 'QGIS + PostgreSQL 실습용 Node.js API 문서',
22     },
23     servers: [
24       {
25         url: 'http://localhost:3000',
26       },
27     ],
28   },
29   // 이 파일(index.js) 안의 JSDoc 주석을 읽어서 스펙 생성
30   apis: [__filename],
31 };
32
33
```

1. 공간정보 프로그래밍

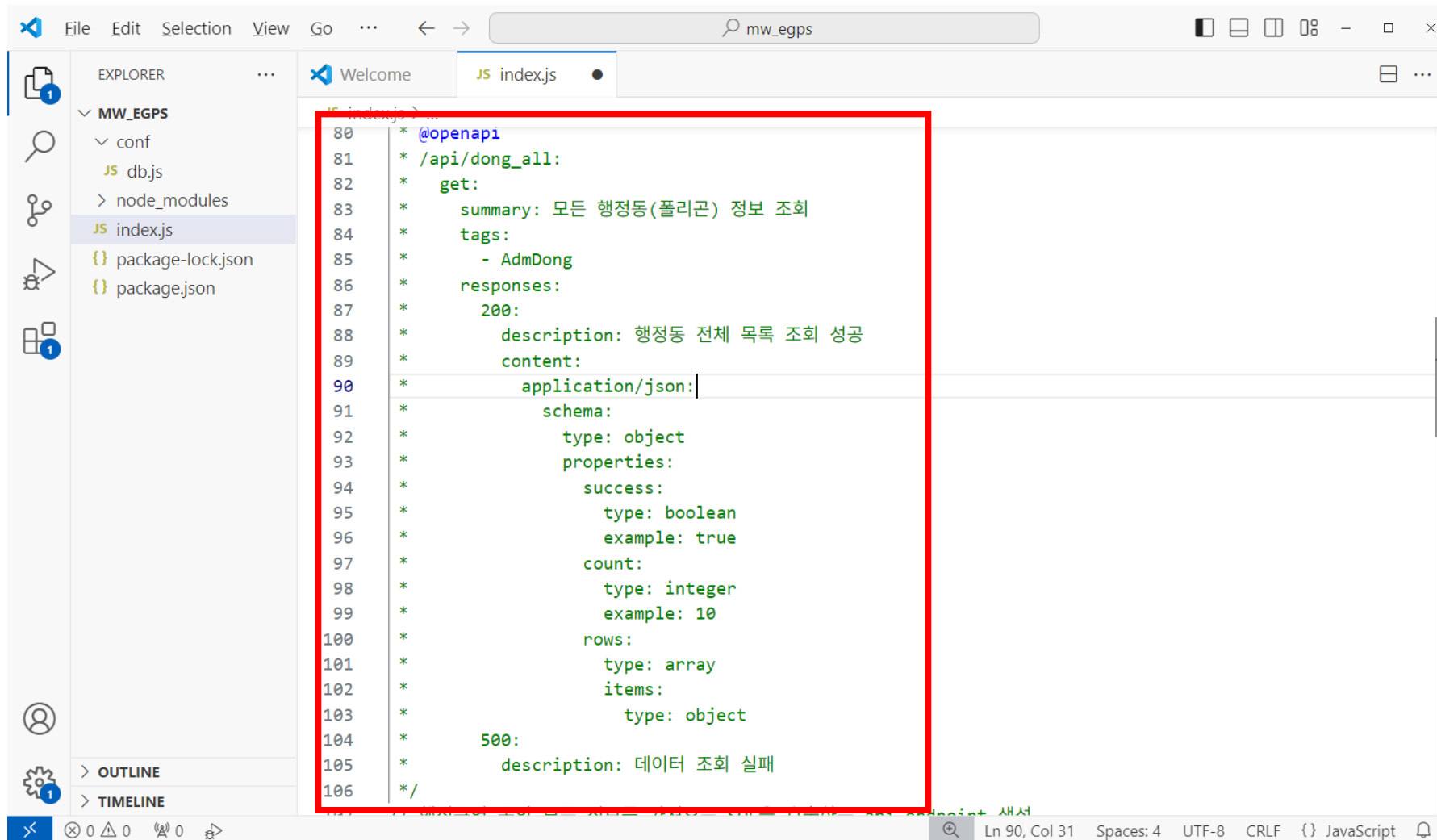
Swagger - express 연동



```
38  /**
39  * @openapi
40  * /db-test:
41  *   get:
42  *     summary: PostgreSQL 연결 및 현재 시간 테스트
43  *     tags:
44  *       - System
45  *     responses:
46  *       200:
47  *         description: DB 연결 성공 시 현재 시간 반환
48  *         content:
49  *           application/json:
50  *             schema:
51  *               type: object
52  *               properties:
53  *                 success:
54  *                   type: boolean
55  *                   example: true
56  *                 dbTime:
57  *                   type: string
58  *                   format: date-time
59  *       500:
60  *         description: DB 연결 또는 쿼리 실패
61  */
62  // DB 연결 테스트용 라우트
63  > app.get('/db-test', async (req, res) => { ...
77  });
```

1. 공간정보 프로그래밍

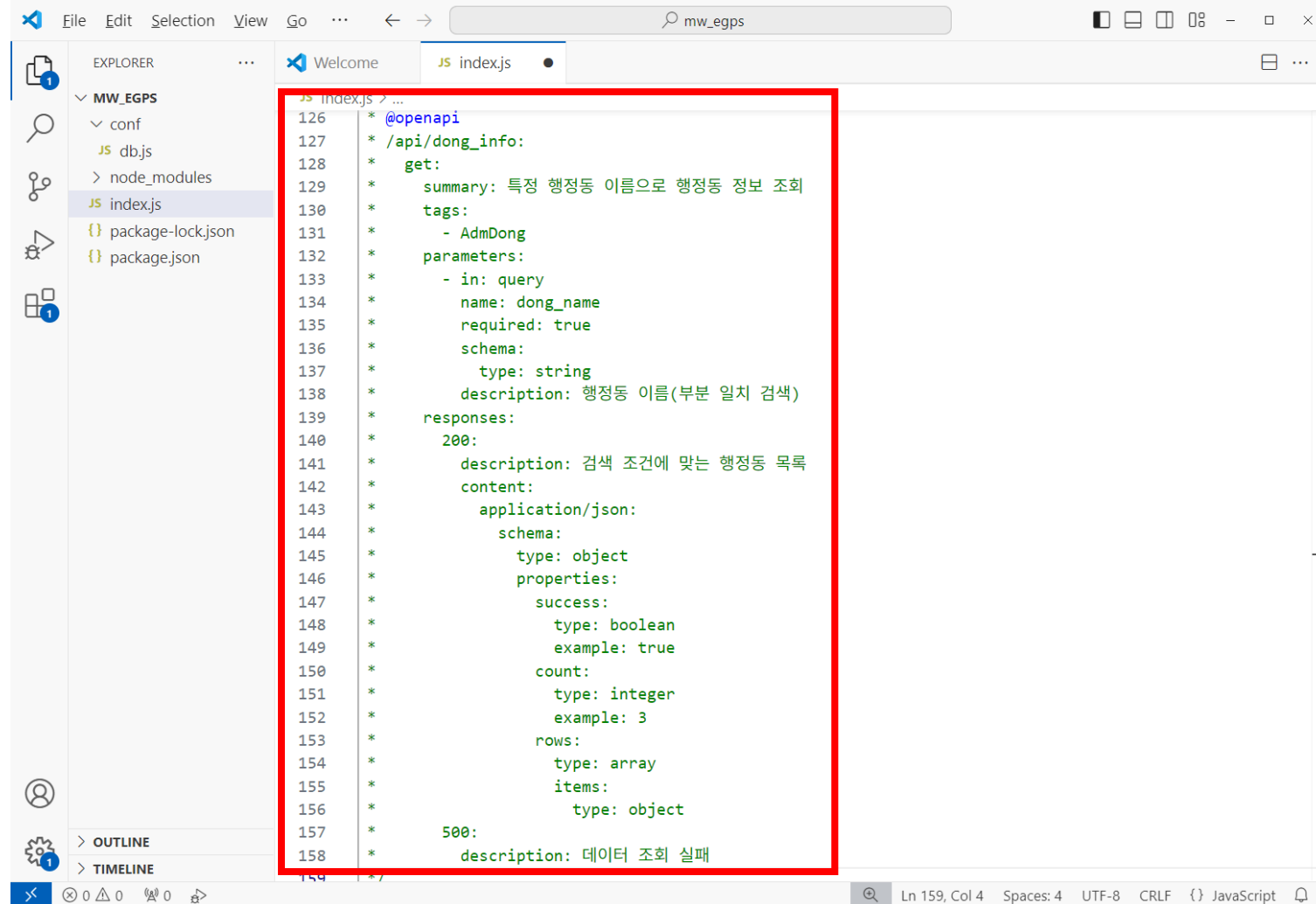
Swagger - express 연동



```
80  * @openapi
81  * /api/dong_all:
82  *   get:
83  *     summary: 모든 행정동(폴리곤) 정보 조회
84  *     tags:
85  *       - AdmDong
86  *     responses:
87  *       200:
88  *         description: 행정동 전체 목록 조회 성공
89  *         content:
90  *           application/json:
91  *             schema:
92  *               type: object
93  *               properties:
94  *                 success:
95  *                   type: boolean
96  *                   example: true
97  *                 count:
98  *                   type: integer
99  *                   example: 10
100  *                 rows:
101  *                   type: array
102  *                   items:
103  *                     type: object
104  *                 500:
105  *                   description: 데이터 조회 실패
106  */
```

1. 공간정보 프로그래밍

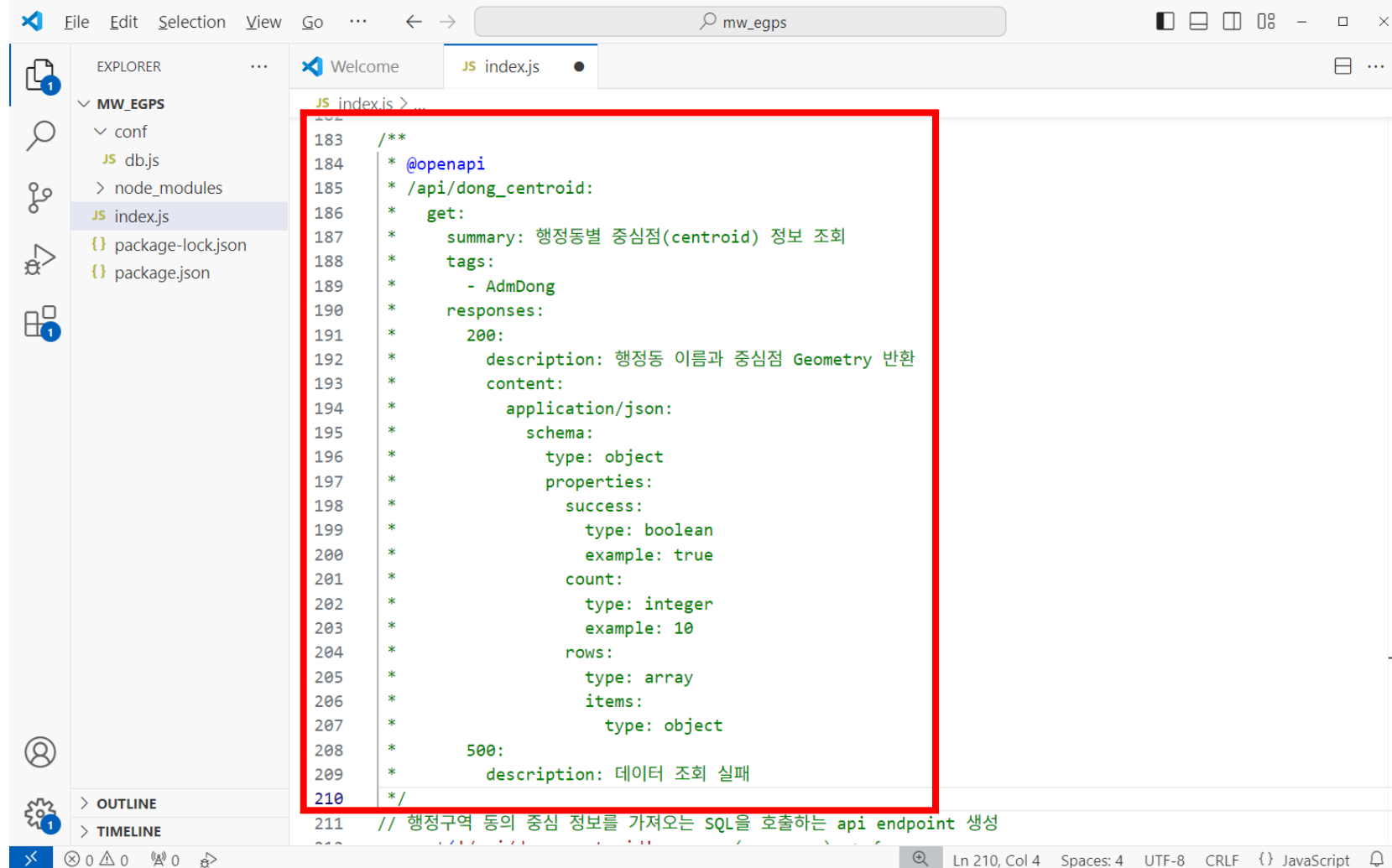
Swagger - express 연동



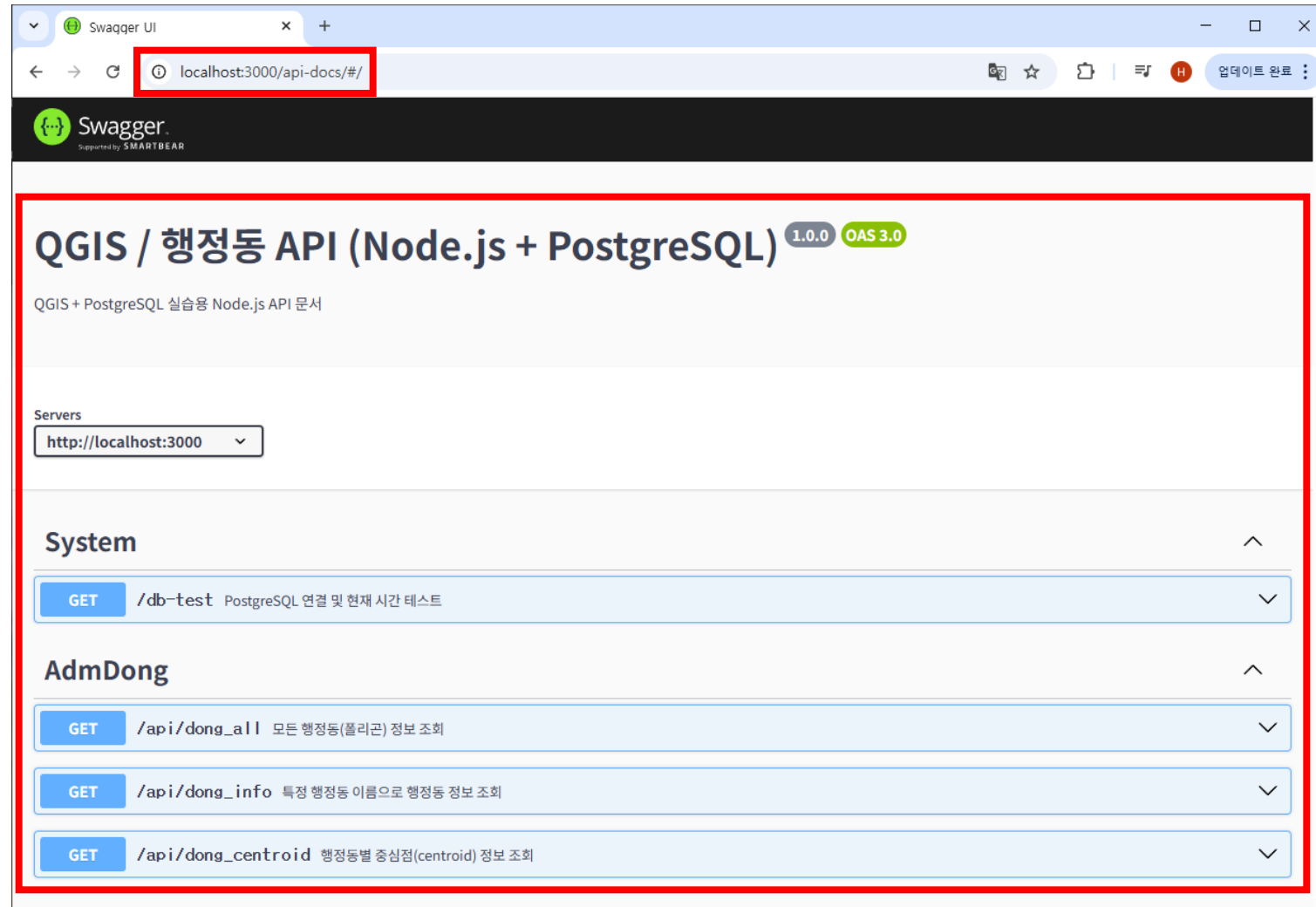
```
126 * @openapi
127 * /api/dong_info:
128 *   get:
129 *     summary: 특정 행정동 이름으로 행정동 정보 조회
130 *     tags:
131 *       - AdmDong
132 *     parameters:
133 *       - in: query
134 *         name: dong_name
135 *         required: true
136 *         schema:
137 *           type: string
138 *           description: 행정동 이름(부분 일치 검색)
139 *     responses:
140 *       200:
141 *         description: 검색 조건에 맞는 행정동 목록
142 *         content:
143 *           application/json:
144 *             schema:
145 *               type: object
146 *               properties:
147 *                 success:
148 *                   type: boolean
149 *                   example: true
150 *                 count:
151 *                   type: integer
152 *                   example: 3
153 *                 rows:
154 *                   type: array
155 *                   items:
156 *                     type: object
157 *       500:
158 *         description: 데이터 조회 실패
```

1. 공간정보 프로그래밍

Swagger - express 연동



```
183 /**
184  * @openapi
185  * /api/dong_centroid:
186  *   get:
187  *     summary: 행정동별 중심점(centroid) 정보 조회
188  *     tags:
189  *       - AdmDong
190  *     responses:
191  *       200:
192  *         description: 행정동 이름과 중심점 Geometry 반환
193  *         content:
194  *           application/json:
195  *             schema:
196  *               type: object
197  *               properties:
198  *                 success:
199  *                   type: boolean
200  *                   example: true
201  *                 count:
202  *                   type: integer
203  *                   example: 10
204  *                 rows:
205  *                   type: array
206  *                   items:
207  *                     type: object
208  *                 500:
209  *                   description: 데이터 조회 실패
210  */
211 // 행정구역 등의 중심 정보를 가져오는 SQL을 호출하는 api endpoint 생성
```



감사합니다

임 호 용

imyong7@hanmail.net
